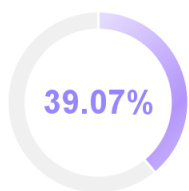


NO. a545a160b74u841a | 2026-04-01 20:05:31

- 题目：基于Django的大学生就业信息共享平台
- 作者：罗展鹏
- 检测所属单位：广东工业大学

📄 论文字符数：42173 📄 论文页数：89 📊 表格数量：27 🖼️ 图片数量：65

检测结果



39.07%

疑似AIGC生成占比

60.93%

人工撰写占比

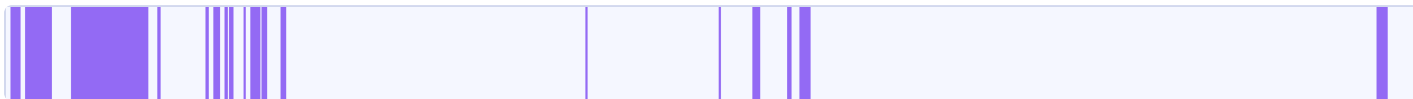
结果分布

序号	章节	疑似AIGC生成文字/章节总字数	疑似AIGC生成章节占比	人工撰写占比
1	摘要	688/690	99.0%	1.0%
2	Abstract	259/349	74.0%	26.0%
3	1. 绪论	3417/3566	95.0%	5.0%
4	2. 系统相关技术概述	2064/2343	88.0%	12.0%
5	3. 系统需求分析	286/3472	8.0%	92.0%
6	4. 系统设计	2898/9008	32.0%	68.0%
7	5. 系统详细设计与实现	1232/7540	16.0%	84.0%
8	6. 系统测试	796/7356	10.0%	90.0%

*注:格式规范的情况下可准确识别章节,若论文中无章节,可能会识别有误。

片段分布

■ 疑似AIGC生成 □ 人工撰写



文字标注

■ 疑似AIGC生成 ■ 人工撰写



基于Django的大学生就业信息共享平台

廣東工業大學

本科毕业设计(论文)

基于Django的大学生就业信息共享平台

学 院: 先进制造学院

专 业: 人工智能

年级班别: 2022级(3)班

学 号: 312009223

学生姓名: 罗展鹏

指导教师: 康培培

2026 年 4月

摘要

针对当前高校毕业生人数逐年递增、就业竞争加剧,且大学生面临就业信息分散、经验支撑不足、求职与招聘信息匹配效率低等问题,本文设计了一款基于Django框架的大学生就业信息资源共享平台,采用前后端分离架构,结合Python编程语言、Vue.js前端技术、数据库管理技术及软件工程方法论,构建高效、安全的就业信息共享平

台。

平台围绕“学生-企业-系统管理”三类用户需求划分核心功能模块：学生模块支持用户登录注册、简历上传编辑、就业经验分享、招聘信息精准筛选及求职实时提醒，解决学生获取就业资源碎片化、经验支撑不足的痛点；企业模块聚焦资质审核与基础招聘功能，实现企业合规入驻与求职者简历定向查看，保障招聘信息真实性；系统管理模块通过用户账号与权限管控、招聘及发帖内容合规审核、公告发布管理，维护平台信息安全与运营秩序，同时助力打通“经验学习 - 岗位匹配 - 投递应聘”的服务闭环。

开发过程中，首先完成需求分析与整体方案设计，明确模块划分原则与数据交互逻辑；其次基于Django REST Framework构建后端API接口，设计MySQL数据库存储用户、简历、招聘等核心数据，结合Vue.js完成前端页面开发；最后通过功能模块测试、组装测试及需求验证，确保平台各功能稳定运行——测试结果表明，平台可实现就业信息高效共享与精准匹配，满足学生求职、企业招聘及平台管理的多元化需求，为高校就业服务数字化提供实践参考。

本文的创新点在于将“信息共享”与“合规审核”深度结合，在提升就业信息流通效率的同时，通过多层级审核机制保障信息质量；技术层面采用前后端分离架构，降低系统耦合度，便于后续功能扩展与维护。

关键词：Django框架，MySQL数据库，Vue.js，前后端分离，就业信息平台

Abstract

In response to the increasing number of college graduates and intensified job competition, as well as the issues faced by college students such as scattered employment information, lack of experience support, and low efficiency in matching job seekers with job postings, this paper designs a college student employment information sharing platform based on the Django framework. It adopts a front-end and back-end separation architecture, combining Python programming language, Vue.js front-end technology, database management technology, and software engineering methodology to build an efficient and secure employment information sharing platform.

The platform is designed around three user categories: students, enterprises, and system administrators, with core modules tailored to their needs. The student module supports login registration, resume editing, experience sharing, targeted job search alerts, and precise job postings filtering, addressing the challenges of fragmented employment resources and insufficient experiential support for students. The enterprise module focuses on qualification verification and basic recruitment functions, enabling compliant business registration and targeted resume viewing for job seekers, ensuring the authenticity of job postings. The system administration module maintains platform security and operational order through user account management, content compliance review, and announcement management, while facilitating the seamless service loop of "experience learning-position matching-application submission".

During the development process, first complete the requirement analysis and overall solution design, clarifying the principles of module division and data interaction logic; second, build the backend API interfaces based on the Django REST Framework, design the MySQL database to store core

data such as users, resumes, and job postings, and integrate with Vue.js to develop the frontend pages; finally, ensure the stable operation of all platform functions through functional module testing, assembly testing, and requirement validation—test results show that the platform can achieve efficient sharing and precise matching of employment information, meeting diverse needs of students seeking jobs, companies recruiting, and platform management, providing practical references for the digitalization of university employment services.

The innovation of this paper is to combine “information sharing” and “compliance audit” deeply, which can improve the efficiency of employment information circulation and ensure the quality of information through multi-level audit mechanism.

Keywords: Django framework, MySQL database, Vue.js, Front-end and back-end separation, Employment information platform

目录

- 1. 绪论1
 - 1.1. 研究背景1
 - 1.2. 国内外研究现状2
 - 1.2.1. 国内研究现状2
 - 1.2.2. 国外研究现状3
 - 1.3. 论文的主要研究内容3
 - 1.4. 论文的组织结构4
- 2. 系统相关技术概述5
 - 2.1. Django框架5
 - 2.2. Vue框架5
 - 2.3. 非关系型数据库Redis6
 - 2.4. JWT权限验证6
 - 2.5. MySQL数据库7
 - 2.6. Django REST Framework7
- 3. 系统需求分析8
 - 3.1. 业务需求概述8
 - 3.2. 系统功能性需求分析8
 - 3.2.1. 系统整体需求分析8
 - 3.2.2. 用户端模块需求分析9
 - 3.2.3. 企业端模块需求分析12
 - 3.2.4. 管理员端模块需求分析14
 - 3.3. 系统非功能性需求分析15

3.3.1. 系统性能需求	15
3.3.2. 系统安全性需求	15
3.3.3. 系统可扩展性需求	16
3.4. 本章小结	16
4. 系统设计	17
4.1. 系统架构设计	17
4.2. 系统功能模块设计	18
4.2.1. 学生求职者用户端功能模块设计	18
4.2.2. 企业端功能模块设计	20
4.2.3. 管理员端功能模块设计	22
4.3. 系统数据库设计	23
4.3.1. 数据库概念模型设计	23
4.3.2. 数据库物理模型设计	27
4.4. 本章小结	34
5. 系统详细设计与实现	35
5.1. 系统技术分析	35
5.2. 系统功能模块详细设计与实现	35
5.2.1. 用户注册登录模块	35
5.2.2. 简历管理模块	37
5.2.3. 经验社区模块的详细设计与实现	39
5.2.4. 职位筛选和简历投递申请模块	41
5.2.5. 聊天室模块	43
5.2.6. 消息通知模块	45
5.2.7. 企业端职位发布与管理模块	46
5.2.8. 企业端招聘过程管理模块	48
5.2.9. 招聘数据统计模块	50
5.2.10. 管理员模块	51
5.3. 本章小结	53
6. 系统测试	54
6.1. 系统的测试环境	54
6.2. 系统功能测试	54
6.2.1. 用户注册登录功能模块测试	55
6.2.2. 简历管理模块测试	58
6.2.3. 经验分享社区模块测试	59

6.2.4. 职位筛选与投递模块测试63

6.2.5. 聊天室模块测试66

6.2.6. 通知模块测试67

6.2.7. 认证功能测试69

6.2.8. 企业端功能测试70

6.2.9. 管理员功能测试73

6.2.10. 三端主界面展示75

6.3. 本章小结77

总结与展望78

参考文献79

致谢81

1. 绪论

1.1. 研究背景

近年来，随着高等院校毕业生人数逐年递增，我国高校毕业生就业形势愈发严峻，各高校毕业生资源与各企业之间供求关系的不对称现象也愈发明显，众多毕业生面临着就业难的问题。这问题成为了个人、高校、企业、国家都关心的热点问题和难题[1]。据教育部公开数据显示，全国2025届高校毕业生人数达到1222万，总量创历史新高[2]。庞大的毕业生群体与有限的优质岗位资源形成“僧多粥少”的竞争格局，再加上经济结构调整、部分行业波动等等外部因素，大学生就业压力持续加剧，传统就业服务模式已难以适应当今形势下的就业需求。同时，国家高度重视高校毕业生就业问题，《“十四五”促进就业规划》明确提出“实施常态化高校毕业生就业信息服务，精准组织线上线下就业服务活动”，要求高校与社会平台协同发力，推动就业信息、指导资源、招聘服务的整合共享，这为就业服务的模式创新提供了政策导向与战略支撑。

在当前的就业生态中，大学生就业环节存在“信息不对称”与“资源碎片化”问题[3]，制约就业效率和质量。从信息获取层面来看，大学生获取就业信息的渠道呈现“多而杂”的碎片化特征：学校就业网信息更新滞后、覆盖范围有限，多聚焦本地企业和校招专场；综合招聘平台则以社招为主，校招信息分散在不同板块，缺乏针对大学生的精准筛选标签，如“无经验要求”、“实习转正”；企业宣讲会、笔面试等时效性强的信息则多依赖微信群、朋友圈等非正式渠道传播，信息传播率低、时效性短，常导致学生错失关键机会[4]。从经验支撑层面来看，大学生作为从未进入就业市场的新人，对行业选择、简历优化、面试技巧等实操环节存在强烈需求，而现有资源存在明显缺陷：社交平台上的经验分享内容碎片化且真实性难辨，掺杂广告或夸大成分；高校线下就业指导覆盖人数有限，难以针对不同专业、不同求职方向提供个性化建议，导致学生在“互联网校招流程”、“国企面试注意事项”等关键决策环节缺乏可靠参考。从供需匹配层面来看，企业与学生之间的精准对接存在障碍：企业难以快速筛选出符合特定需求的学生，简历筛选成本高；学生则因缺乏对企业真实工作环境、岗位实际需求的了解，易出现“盲目投递”、“入职后短期离职”等问题，而中小企业招聘信息更因传播渠道有限，难以触达目标院校学生，加剧了供需匹配的低效性。

现有就业服务平台未能有效解决上述痛点，存在明显的功能适配缺陷。以智联招聘、BOSS直聘为代表的综合招聘平台，核心服务对象是有工作经验的职场人，校招板块仅为附加功能，且企业之间信息不对称，虚假信息过多[5]；多数高校就业网信息覆盖范围窄、更新频率低，仅支持信息浏览，无法实现学生间的经验互动或企业的精准筛选，用户粘性差；应届生求职网等垂直平台虽提供面经、笔试题库，但未与招聘信息、企业资源打通，形成“信息孤岛”，学生获取经验后仍需跳转至其他平台投递简历，无法形成“经验学习 - 岗位匹配 - 投递应聘”的服务闭环，用户体验割裂。

在此背景下，构建一款聚焦大学生需求的“就业信息共享平台”具有明确的现实意义与应用价值。本文设计的平台整合“招聘信息整合、真实经验分享、精准供需匹配”三大核心功能，为大学生提供更便捷的就业服务，缓解信息获取低效、经验支撑不足的问题，也能帮助企业降低招聘成本、提升匹配效率，同时助力国家就业服务数字化战略落地，为缓解大学生就业压力提供切实可行的技术解决方案。

1.2. 国内外研究现状

1.2.1. 国内研究现状

随着数字化就业服务的发展，国内研究已从最初的对招聘平台信息发布、简历投递等基础功能的探讨，逐步深化至对招聘服务生态、技术架构及社会效应的系统性分析。目前，大多数高校的就业服务工作内容较为繁琐，主要依赖于人力或群发的方式组织就业讲座、办理毕业生就业手续、举办招聘会和宣讲会，以及收集相关就业数据等[1]。研究普遍认为，以大数据和人工智能算法为核心的平台能够显著提升岗位匹配效率，但其算法可能隐含的偏见、对信息真实性保障的不足，以及用户数据安全风险等问题也引发了广泛讨论。特别是在解决大学生求职中的“信息不对称”和“结构性矛盾”方面，当前多数平台尚未能完全实现高效、精准且具有公信力的信息对接，在面向院校、专业差异的个性化服务上仍有提升空间。在技术实现上，尽管主流研究认可一体化、智能化平台的发展方向，但聚焦于采用如Django等特定开源框架进行轻量化、模块化系统设计与开发的实践性探讨相对较少，这为从工程实施视角探究高可用性、易保养且成本可控的就业信息共享平台提供了研究突破口。此外，怎样将平台与高校就业指导工作深度结合，打造一个由学校、企业、学生多方共同参与，包含信息共享、职业服务和决策支持功能的可信环境，已经成为目前研究的关键趋势与共识，也为本研究基于Django框架构建一个符合实际需求、重视信息安全与用户体验的就业信息共享平台指出了实际依据与设计目标。

1.2.2. 国外研究现状

国外研究现状方面，欧美等发达国家和地区由于高等教育市场化程度较高、信息技术应用起步较早，对在线招聘平台及其社会影响的研究已形成了较为成熟的理论谱系与实践批判。其招聘平台领域已形成从基础信息管理向智能化、集成化系统演进的整体趋势，在技术架构上普遍采用分层设计和B/S模式，可结合Spring Boot等现代Web框架构建可扩展平台，支持多角色协同工作流[6]。招聘全流程与人工智能技术深度融合，机器学习算法不仅实现自动化技能评估和实时代码执行，还通过AI生成自定义编程题目以增强评估的客观性和效率[7]；同时，即时通知服务的集成确保了申请状态的实时推送，显著提升沟通效果[6]。在用户体验层面，研究强调参与式设计方法，通过技能调查和可视化工具（如技能比较图表）帮助用户进行自我评估，这种设计结合移动端适配界面，促进了求职者与企业的互动[8]。系统评估通过用户接受度测试验证平台效能，报告显示招聘处理时间减少超40%、用户满意度达90.8%的优化成果[6]，但仍面临数据标准化、算法偏见校正等挑战[7]。未来研究倾向于强化AI与隐私保护的深度融合，进一

步提升平台的智能化和公平性[8]。

1.3. 论文的主要研究内容

本文基于Django和Vue框架构建大学生就业信息共享平台，系统架构采用前后端分离设计，由后台管理系统和前台Web用户端（学生端、企业端）两个主要部分组成。本设计的后台管理系统仅供管理员使用，其功能包括管理员账号登录、用户管理（求职学生账户与企业账户资质审核）、招聘信息真实性合理性审核、经验贴内容管控及就业数据统计等。Web用户端开放对象为学生和企业两类，学生端拥有注册登录、简历创建与管理、招聘信息查询与投递、就业经验贴浏览与分享等功能；企业端具备企业认证、招聘信息发布与管理、学生简历筛选、面试邀约等功能。

本系统所具备的注册登录功能，其主要服务对象为学生与企业用户。当学生有就业信息查询、简历投递需求，或企业有招聘信息发布需求时，需先行完成注册和登录操作，这一流程是两类用户使用平台核心服务的前置步骤。用户信息管理功能分为两部分：管理员可在后台审核企业资质、查看学生基础信息；学生与企业可在各自端内编辑个人/企业资料，确保信息真实性与完整性。

管理员借助招聘信息审核功能，可对企业发布的岗位信息进行合规性校验，避免虚假招聘；经验分享内容管控功能则用于审核学生发布的就业经验帖，保障内容积极正向。学生通过简历管理功能创建简历，结合招聘信息查询功能筛选适配岗位并完成投递；企业依托招聘信息管理功能发布岗位需求，通过简历筛选功能快速定位符合“专业匹配”“实习经历”等条件的学生。此外，后台的数据统计功能为管理员提供平台用户状况、岗位投递量、经验帖互动量等数据查看途径，为就业服务优化提供参考。

1.4. 论文的组织结构

本文共分为六个主要章节，系统性地阐述了大学生就业信息共享平台的完整研发过程。第一章为绪论，重点分析大学生就业市场的背景现状和信息化需求，明确平台建设目标和论文研究内容；第二章为相关技术介绍，详细说明系统开发中采用的关键技术栈，包括Django框架、MySQL数据库及前端Vue框架技术，为后续开发奠定基础；第三章开展系统需求分析，通过用例分析和业务流程梳理，明确学生、企业和管理员三类用户的核心需求，形成完整的需求规格说明；第四章进行系统设计，包括架构设计、功能模块划分和数据库设计，其中功能模块涵盖招聘管理、简历投递、社区互动等核心功能；第五章实现系统核心功能，基于MVC模式完成前后端开发，还实现聊天室和实时消息通知等特色功能；第六章进行系统测试，通过功能测试和用户体验测试，验证系统的稳定性和实用性；最后的总结与展望，归纳研究成果，分析系统局限性，并提出后续优化方向。各章节内容紧密衔接，构成完整的平台研发体系。

2. 系统相关技术概述

本章将介绍大学生就业信息共享平台系统开发中设计到的基础知识和技术背景，包括Django框架、Vue、MySQL、Redis和JWT权限验证等，这些技术高效地支持着本系统的研发工作。

2.1. Django框架

Django是一个用Python语言编写的开源应用框架，遵循MTV架构[9]，以其“开箱即用”的特性和强大的可扩展性而闻名，能够快速开发大型网站，是当前较为流行的一种Web开发框架。

Django框架的核心架构包含以下几个关键组件：ORM[10]（对象关系映射）系统允许开发者使用Python类来定义数据模型，自动生成数据库表结构，可直接通过模型操作数据库；模板引擎支持动态HTML页面生成；视图层负责处

理业务逻辑；路由系统提供灵活的URL映射机制，这种分层的架构让代码结构清晰且便于维护和扩展。

另外，Django提供了全面的防护措施，包括CSRF[11]（跨站请求伪造）保护、XSS[12]（跨站脚本）防护、SQL注入预防[13]等。其内置的用户认证系统支持用户注册、登录、权限管理等功能，这些功能大大简化了开发流程。

Django的还有一个特点是其强大的管理后台，本平台的管理员端也是直接由其内置的管理后台修改而来，它可以自动生成数据管理界面，支持数据的增删改查操作。并且Django的中间件机制、缓存系统、国际化支持等特性，使它成为开发复杂Web应用的一个理想选择。

2.2. Vue框架

Vue.js[14]是一套用于构建用户界面的渐进式JavaScript框架，以其轻量级、高性能和易用性而受到广泛欢迎，Vue采用组件化开发模式，支持声明式渲染和响应式数据绑定，能够高效构建交互式Web应用。

Vue的核心特性包括虚拟DOM技术，通过差异比对算法优化页面渲染性能；其响应式系统能自动追踪数据依赖并在数据变化时更新视图，还拥有组件系统支持可复用的UI组件开发。

Vue框架的生态系统十分丰富，包含Vue Router用于前端路由管理、Vuex用于状态管理、Vue CLI用于项目脚手架搭建。

在性能优化方面，Vue提供了计算属性、侦听器、条件渲染等特性，帮助开发者编写高效的代码，其服务端渲染（SSR）能力也支持更好的SEO优化和首屏加载性能。

2.3. 非关系型数据库Redis

Redis[15]是一种基于内存存储原理的开源数据库系统。它使用内存作为主要的数据存储介质，通过将数据存储在内存中，提供快速的读写操作，支持多种数据结构，包括字符串、哈希、列表、集合、有序集合等。

Redis通过RDB快照和AOF日志两种方式实现数据持久化；丰富的数据类型，支持复杂的数据结构操作；内置复制功能，支持主从同步和数据备份。

在应用场景上，Redis常用于缓存系统，减轻后端数据库压力；会话存储，管理用户登录状态；消息队列，支持发布/订阅模式；实时排行榜，利用有序集合实现排序功能。其原子操作和事务支持也保证了数据的一致性。Redis的高可用方案包括主从复制、哨兵模式和集群模式，能够满足不同规模应用的需求。

2.4. JWT权限验证

JWT（JSON Web Token）[16]是一种基于JSON的开放标准（RFC 7519），用于在各方之间安全地传输信息作为JSON对象，这种令牌通常用于身份验证和授权场景，在分布式系统和微服务架构中广泛应用。

JWT的组成结构包括头部（Header）、载荷（Payload）和签名（Signature）三部分。头部包含令牌类型和签名算法信息；载荷包含声明语句，像用户身份、权限等；签名用于验证消息的完整性和真实性。

JWT的工作流程包含令牌生成、传输和验证三个环节。用户登录后，服务器生成JWT并返回给客户端；客户端在后续请求中携带JWT；服务器验证JWT的合法性并处理请求，这种无状态的设计使得JWT适合RESTful API的认证需求。

与传统的Session认证相比，JWT具有跨域支持、无状态服务、移动端友好等优势。然而，也需要考虑令牌失效、安全存储等问题。在实际应用中，通常需要结合HTTPS、合理的过期时间设置等安全措施来确保JWT的安全性。

2.5. MySQL数据库

MySQL是全球最流行的开源关系型数据库管理系统，以其稳定性、可靠性和高性能而闻名。作为关系型数据库的代表，MySQL支持标准的SQL语言，提供完整的事务处理能力。

MySQL的架构特点包括存储引擎机制，支持InnoDB、MyISAM等多种存储引擎；索引优化，通过B+树索引提高查询效率；事务支持，满足ACID特性要求；复制功能，支持主从复制和读写分离。

在数据安全方面，MySQL提供完善的权限管理系统，支持用户认证和访问控制；数据加密功能，保护敏感信息；备份恢复机制，确保数据可靠性。其监控工具和性能优化功能也帮助管理员维护数据库的健康状态。

MySQL适用于各种规模的应用场景，从小型网站到大型企业系统都能提供良好的支持。其与各种编程语言的兼容性和丰富的工具生态，是Web开发的首选数据库。

2.6. Django REST Framework

Django REST Framework（简称DRF）是基于Django的强大API开发工具包，能够快速构建符合RESTful 风格的接口，支持前后端分离、自动生成API文档，并提供完善的认证、权限和序列化机制。

DRF的核心功能包括序列化器，用于数据验证和转换；基于类的视图，支持API端点的快速开发；认证和权限系统，提供灵活的访问控制；路由系统，自动生成API URL；分页和过滤，支持大数据集的高效处理。

DRF与Django生态的深度集成，可以直接使用Django的模型和认证系统；拥有丰富的功能模块，能减少重复代码编写；提供可浏览的API界面，方便测试和文档查看；并且拥有灵活的扩展机制，支持自定义功能开发。

3. 系统需求分析

本章围绕基于Django的大学生就业信息共享平台进行需求分析，重点从功能性和非功能性两个维度进行研究和探讨。先明确平台开发的实际背景和用户的真实业务需求，通过用例的分析和模块划分，明确功能性需求和非功能性需求，其中，面向功能的需求分析对系统的各部分最终需要实现哪些具体的功能进行详细分析，是本章的重中之重。

3.1. 业务需求概述

本项目最终目的是构建一个功能相对完整的大学生就业信息共享平台系统，系统采用Django+Vue前后端分离技术体系搭建核心框架和实现主要功能。整合“招聘信息聚合、经验共享、实时聊天”等功能，为学生、企业、管理员三类用户提供一站式服务。学生端支持简历管理、岗位投递、经验分享；企业端实现招聘发布、简历筛选、面试邀约；管理员端负责用户审核、内容监管。平台通过多级审核机制保障信息真实性与安全性，旨在打通“经验学习——岗位匹配——投递入职”的服务闭环，提升就业服务效率。

3.2. 系统功能性需求分析

3.2.1. 系统整体需求分析

本平台以“学生——企业——管理员”交互为核心，构建一体化的就业信息共享平台。学生可注册登录、创建管理简历、发布经验贴、筛选简历、投递简历；企业通过资质审核后可发布招聘、简历筛选、通知笔面试、发送邀约；管理员则是负责用户资质审核、内容监管、权限管理、公告发布，确保平台合规运行。

系统整体用例图如图3.1所示。



图3.1 大学生就业信息共享平台的整体系统用例图

此系统整体功能架构采用前后端分离设计，前端使用Vue.js实现交互界面，后端通过Django REST Framework提供API接口；数据存储采用MySQL关系型数据库，和Redis缓存配合来提升查询性能；还通过模块化设计实现高内聚低耦合以支持后续的功能扩展和维护。

3.2.2. 用户端模块需求分析

用户端面向的是求职学生，系统提供完整的就业服务功能，图3.2呈现了用户端模块的功能结构图，图3.3呈现了用户端模块详细用例图。

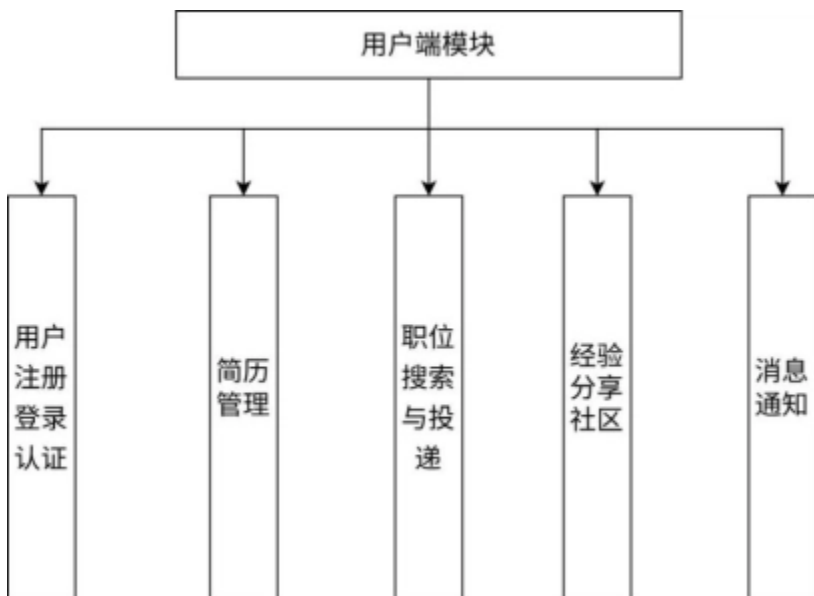
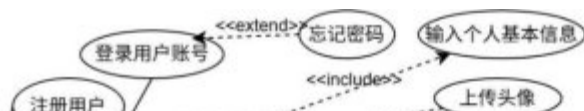


图3.2 用户端模块功能结构图



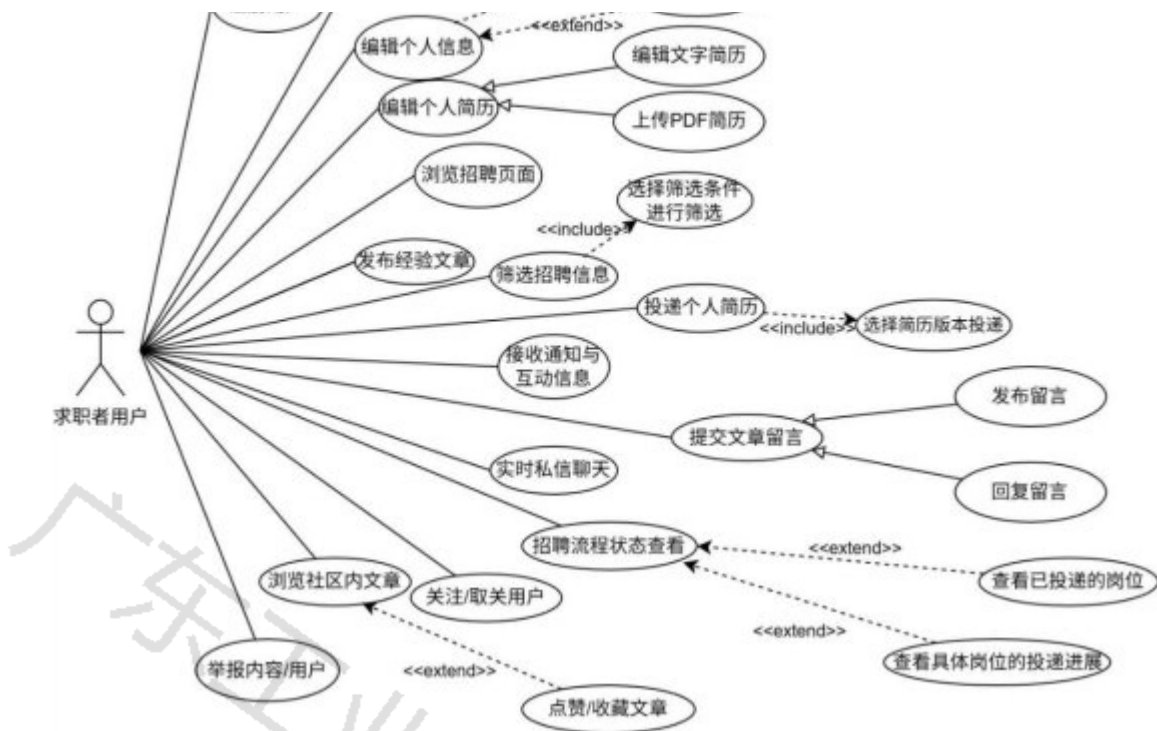


图3.3 用户端模块用例图

1. 用户注册登录认证模块

- (1) 实现学生用户的账号注册功能，注册阶段只需要输入唯一用户名，用户邮箱，并且设置密码；学籍信息、专业背景等基础数据在注册成功后可以登录账户后再补充；
- (2) 提供密码重置和个人信息修改等账户管理功能；
- (3) 集成JWT令牌认证实现会话状态保持和自动登录。

2. 简历管理模块

- (1) 求职学生账户支持简历的创建和修改编辑，简历内容包括教育经历、项目/校园/实习/工作经历、奖项、技能/特长、求职意向、自我评价等内容，系统提供富文本编辑器；
- (2) 简历可上传PDF版本，在后续投递简历时可以将文字简历和PDF简历一并投递；
- (3) 实现简历多版本的存储和管理。

3. 职位搜索与投递模块

- (1) 实现多筛选条件的招聘信息检索功能，支持按职位名称/关键词、公司行业/名称、工作地点、薪资范围、工作性质（如全职/实习）等条件的筛选；
- (2) 投递流程为：选择想要投递的岗位进行申请，然后选择已经编辑好的简历版本，然后进行投递确认；
- (3) 投递后个人“我的申请”页面可以查看已申请的岗位，有投递记录管理，显示已投递的流程信息；
- (4) 支持职位收藏功能。

4. 经验分享社区模块

- (1) 提供面经、笔经和一些吐槽或推荐贴的发布功能；
- (2) 支持发布的内容按照自定义标签分类；

- (3) 实现发布的文章帖子的点赞、评论、收藏等互动功能；
- (4) 发布的内容需要经过机器审核或是人工审核，确保信息的真实性和合规性。

5. 聊天室模块

- (1) 求职者用户端和企业端都设置了聊天室模块，支持实时的聊天功能；
- (2) 聊天室支持文件和图片的发送和接收；
- (3) 聊天室消息会同步至接收方的消息通知模块。

6. 消息通知模块

- (1) 求职用户在投递的简历被处理筛选后，会及时进行反馈、当企业端发起面试邀约后，邀约信息也会实时推送；
- (2) 在文章评论被回复、与平台内其他用户进行私信交流、与企业进行沟通的消息也会实时进行提醒；
- (3) 当用户发布的文章被点赞和评论后，系统也会发送互动消息提醒；
- (4) 在管理员处理举报信息后，用户举报或被举报反馈会通过消息通知模块发送到通知栏。

3.2.3. 企业端模块需求分析

本平台的企业端为招聘企业提供全流程人才选拔功能，企业端模块功能结构图如图3.4所示，企业端模块用例图如图3.5所示。

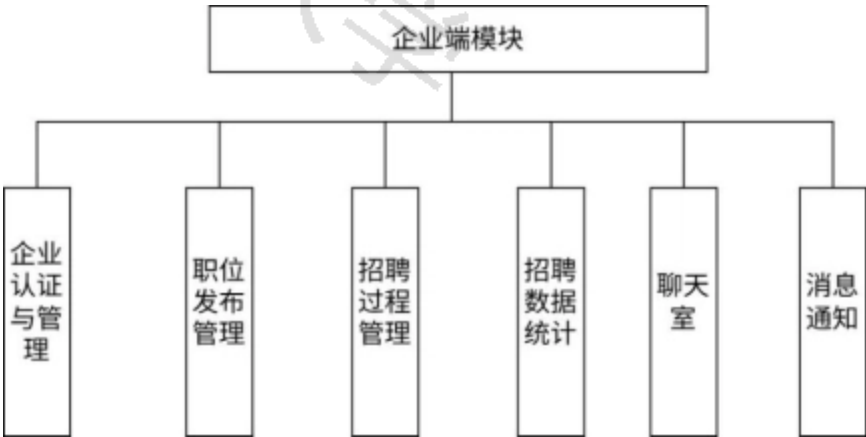


图3.4 企业端模块功能结构图



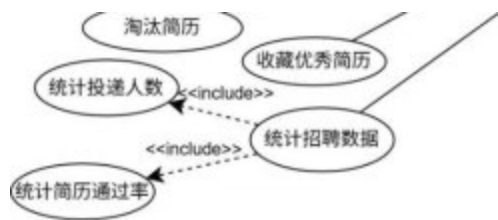


图3.5 企业端模块用例图

1. 企业认证与管理模块

- (1) 企业资质审核流程，需要提交营业执照、公司介绍等必要材料；
- (2) 认证流程和状态的实时查询和通知；
- (3) 企业信息管理，支持基本资料的修改和认证信息的更新。

2. 职位信息发布管理模块

- (1) 职位信息发布和修改功能，包含职位名称、工作地点、工作内容、职责要求、薪资待遇等内容；
- (2) 职位分类管理，支持自定义的职位标签（后续优化扩展思路考虑在发布招聘信息后系统根据标签对求职者进行偏好投放，暂时只实现标签的功能）；
- (3) 职位状态管理，如已发布、草稿、已下架等状态。

3. 招聘过程管理模块

- (1) 简历搜索功能，支持按学历、专业、是否有实习经历等条件筛选；
- (2) 人才库的建设，收藏优秀候选人的简历；
- (3) 设置笔试面试邀约功能，并会显示招聘环节信息，通过通知模块向求职者发送提醒消息，便于招聘环节时间协调和确认。

4. 招聘数据统计模块

- (1) 设置招聘数据统计面板，展示企业招聘进展，包括收到的简历数、初筛通过率等；
- (2) 向求职者发起的笔试/面试邀约总数以及面试通过率。

5. 聊天室模块

- (1) 企业端和求职者用户端一样，都设置了聊天室模块，支持实时的聊天功能；
- (2) 聊天室支持文件的发送和接收；
- (3) 聊天室消息会同步至接收方的消息通知模块；
- (4) 聊天室实时显示发出的消息是否已读。

6. 消息通知模块

- (1) 当求职者投递简历到企业端时，系统会发送通知，接收投递的简历；
- (2) 允许与求职者进行实时聊天，聊天信息也能实时提醒；
- (3) 管理员发布的公告或是举报反馈也同步到对应企业用户通知栏中。

3.2.4. 管理员端模块需求分析

管理员端承担着系统运维和内容监管的职责，在这个系统中起到重要作用，管理员端模块功能结构图如图3.6所示，管理员端模块用例图如图3.7所示。

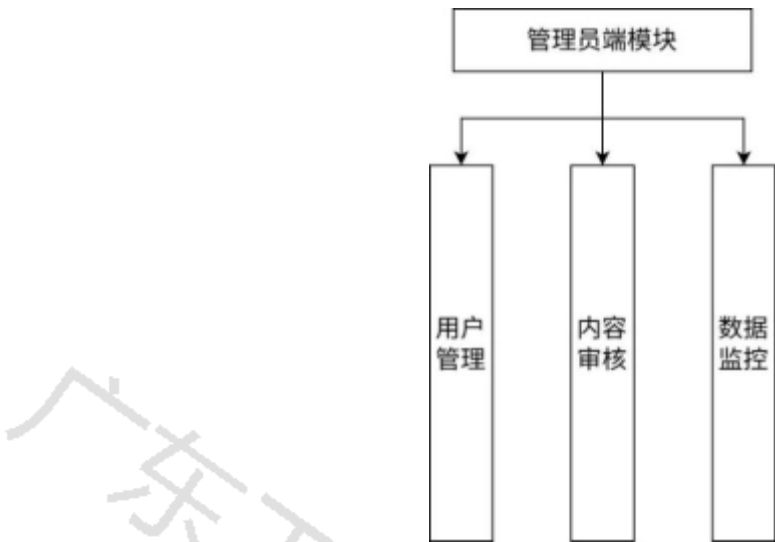


图3.6 管理员端模块功能结构图

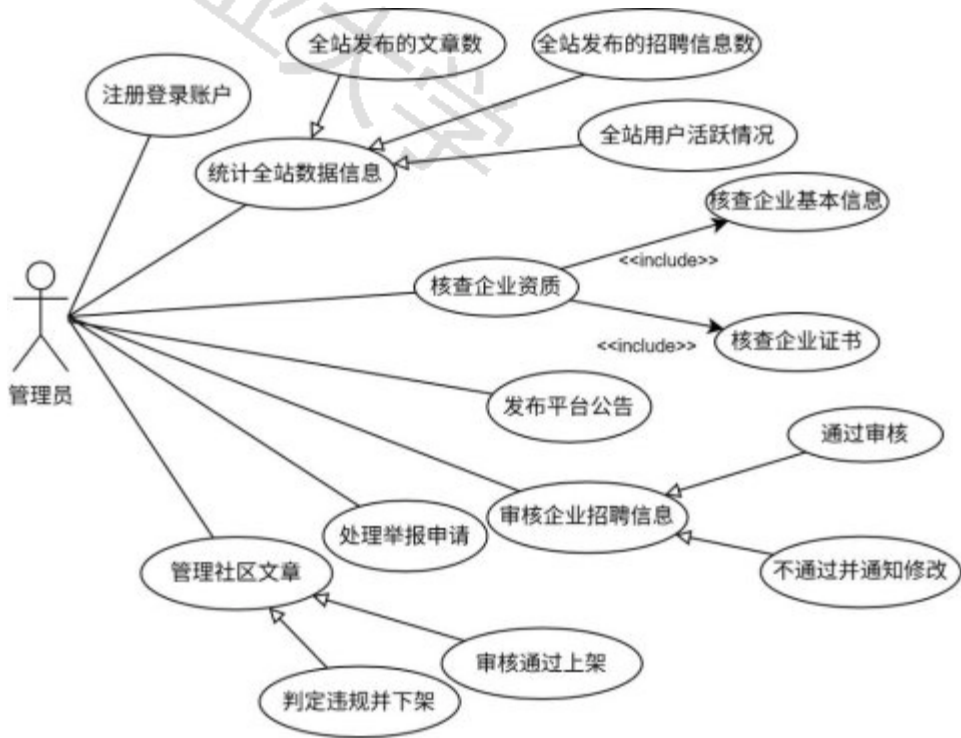


图3.7 管理员端模块用例图

1. 用户管理模块

- (1) 用户账号审核和权限分配，如企业账号资质的审核；
- (2) 账号冻结和解封操作；
- (3) 用户数据统计和分析面板。

2. 内容审核模块

- (1) 对企业发布的招聘信息进行真实性核查，并支持求职者后续举报或反馈，以便对真实情况进行追踪；
- (2) 经验分享内容合规性审查，对言辞过激，虚假分享内容进行下架，对内容发布者进行提示；
- (3) 支持敏感词过滤和屏蔽功能；
- (4) 用户举报信息处理和反馈功能。

3. 数据监控模块

- (1) 系统运行状态实时监控功能，对出现的异常情况进行直观的提示和反馈；
- (2) 用户活跃度和业务数据的统计；
- (3) 设置安全事件日志分析和报警功能；
- (4) 数据备份和恢复管理功能。

3.3. 系统非功能性需求分析

除了系统的功能性需求外，也要满足以下的非功能性需求，保证服务质量。

3.3.1. 系统性能需求

- (1) 在响应时间上，要求核心页面，如用户登陆后的主页，经验社区主页等页面加载时间不超过3秒，API接口的响应时间控制在500毫秒以下；
- (2) 系统同时在线人数较多的情况需要有并发需求，需要支持每秒500个并发用户访问；
- (3) 使用人数多，数据量也会大，需要支持万级用户数据和十万级业务数据存储。

3.3.2. 系统安全性需求

- (1) 用户密码采用加密存储，一些敏感数据要全程加密传输；
- (2) 要明确划分角色的权限，用户隐私信息不可以被其他人查看，并且禁止越权访问和操作；
- (3) 为了能够在发生突发事件进行溯源，系统还要记录关键操作日志，；
- (4) 为应对攻击，系统要具备基础的SQL注入，XSS攻击等常见Web攻击的防护能力。

3.3.3. 系统可扩展性需求

- (1) 系统采用模块化设计，各功能模块低耦合高内聚，支持独立的升级和扩展；
- (2) 接口使用标准化的RESTful API设计，支持多终端输入；
- (3) 数据库需要支持读写分离和分库分表等能力以应对越来越多的数据量和之后的扩展需求。

3.4. 本章小结

本章详细分析了基于Django的大学生就业信息共享平台的需求。功能层面，通过模块化分析明确了学生端、企业端和管理员端的具体需求，形成较完整的功能说明；在非功能部分，从性能、安全、可扩展性等角度制定了实现方向。需求分析这一章为后续系统设计和实现提供了依据，确保平台建设能够真正解决大学生就业过程中的实际问题。

4. 系统设计

在系统的需求分析之后，本文将开展系统的总体设计工作，系统设计主要包括系统架构设计、功能模块设计和数据库概念设计三个方面。通过整体的架构规划、系统的模块划分和数据库的模型设计，平台才能够稳定支持学生、企业、管理员三类用户的交互需求，以此来实现本平台的核心目标。

4.1. 系统架构设计

本平台采用的是前后端分离的架构设计，提升系统的可扩展性、维护性和性能；前端使用Vue.js框架构建用户界面，后端使用基于Django REST Framework (DRF) 提供的RESTful API接口，数据库采用的是MySQL存储主要数据，且集成Redis作为缓存中间件提升优化查询效率。系统整体架构将其分为用户层、表示层、接口层、业务逻辑层、数据访问层和数据存储层，实现高内聚低耦合的设计原则。系统总体架构设计图如图4.1所示。

架构层次设计如下：

用户层是系统的最外层，直接面向三类用户群体：求职学生、招聘企业和系统管理员，用户可以通过浏览器访问，但本平台对移动端暂时不做适配性的设置；

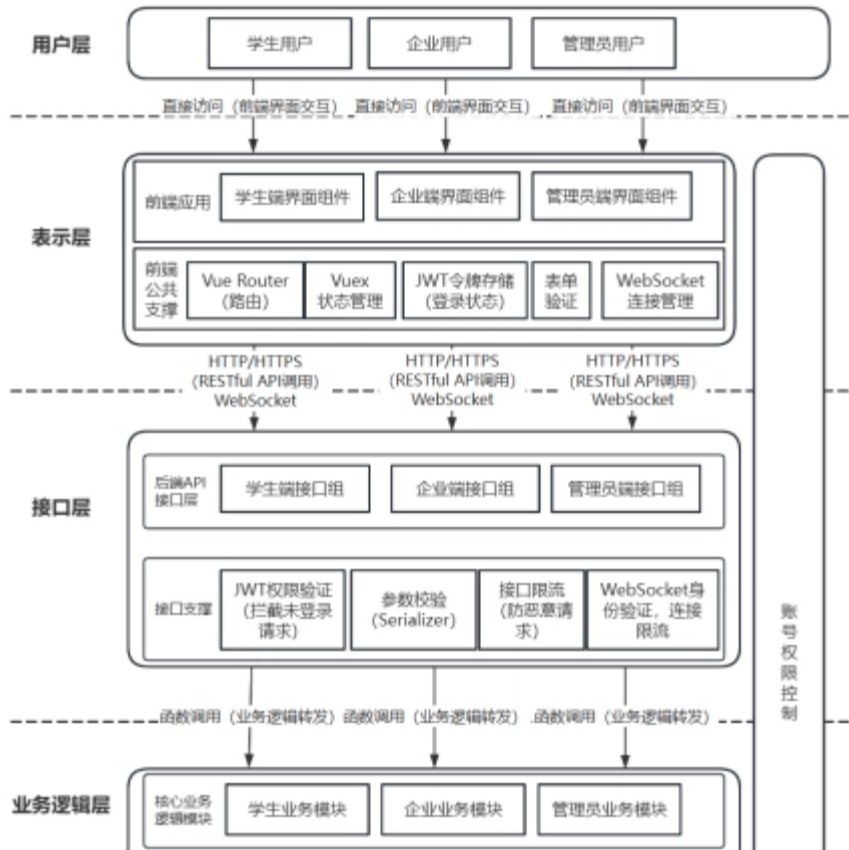
表示层采用Vue.js框架实现，负责用户界面的渲染和交互逻辑，采用组件化的开发方法，将界面分为可复用的模块，比如导航栏，页脚，搜索框，简历编辑器等。Vue Router管理前端路由，Vuex进行状态管理。

接口层采用Django REST Framework构建，提供统一的API网关。所有的前后端数据交互都通过RESTful API完成；接口的设计大部分遵循REST原则，使用HTTP状态码表达操作结果。

业务逻辑层包含学生求职者服务、企业服务和管理员服务三大业务模块。各服务之间通过定义接口进行通信，避免直接依赖，便于后续的扩展和单元测试。DRF提供序列化、权限控制和API路由管理，确保数据交互的安全性和一致性。

数据访问层基于Django的ORM实现，提供统一的数据操作接口。这一层Django已经自动封装了所有数据库访问细节，开发时无需关心具体的数据存储方式，只专注于业务逻辑的实现。

数据存储层用的是混合存储方式：MySQL作为主数据库，存储用户信息、简历数据、职位信息等结构化数据；Redis作为缓存数据库，存储会话信息、热点数据和临时数据。



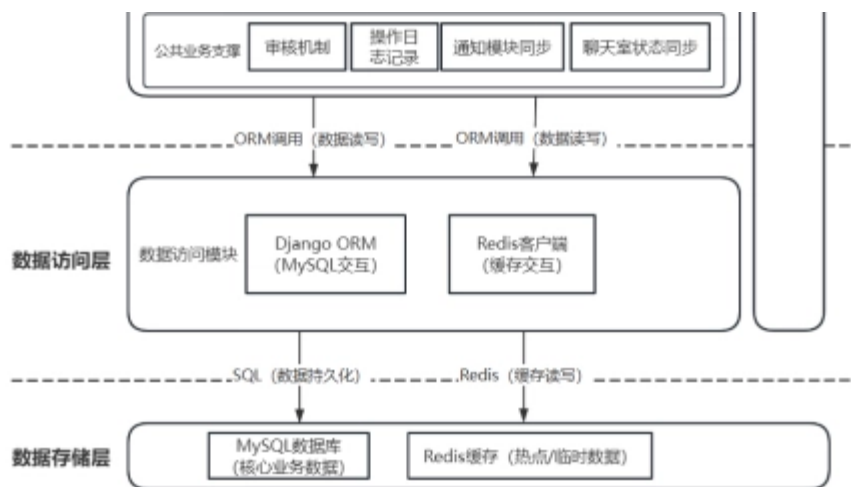


图4.1 系统总体架构设计图

4.2. 系统功能模块设计

4.2.1. 学生求职者用户端功能模块设计

学生求职者用户端模块聚焦求职学生的核心需求，提供完整的就业服务功能，功能模块结构如图4.2所示。

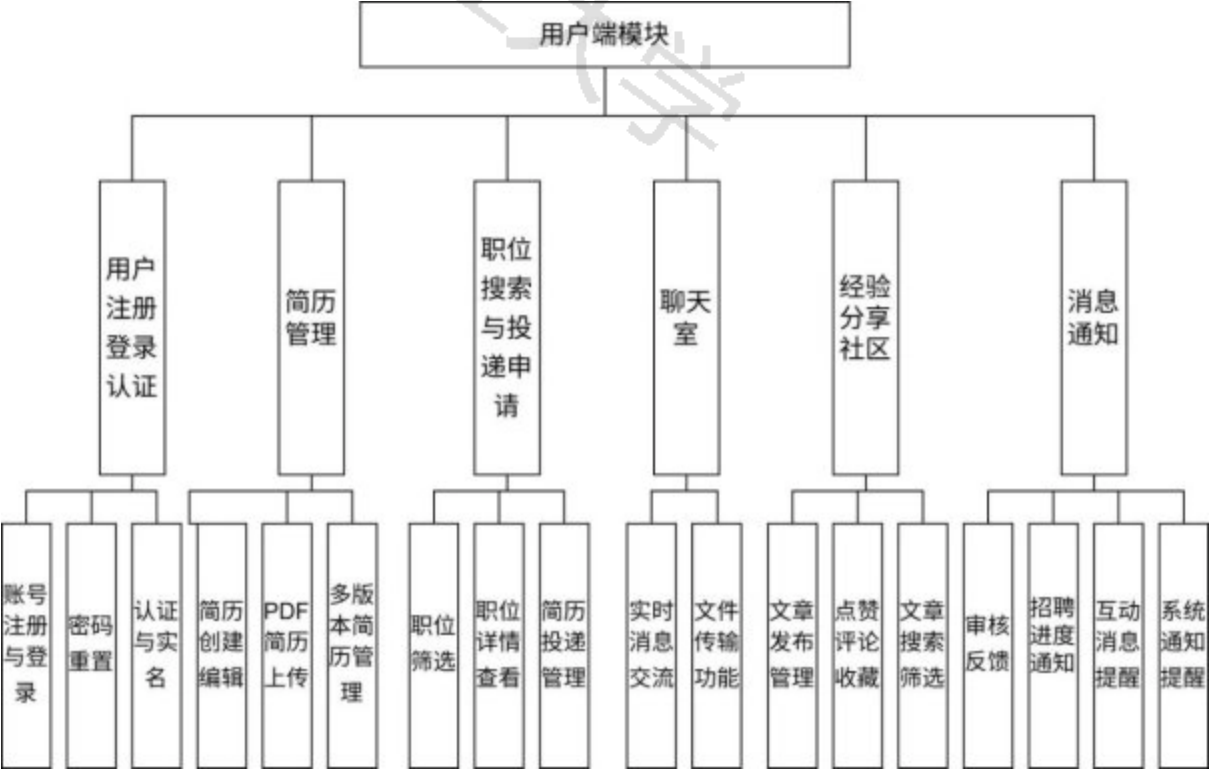


图4.2 学生求职者用户端功能模块结构图

1. 用户注册登录认证模块

采用基于JWT的安全认证机制。学生注册时需提供用户名、邮箱和密码，系统通过邮箱验证来确认用户真实性，注册后的后续认证可扩展采用学信网学籍认证，确保学生求职者实名化。密码采用Django内置算法加密存储，防止密码泄露。登陆成功后，系统生成包含用户ID和权限信息的JWT令牌，有效期为24小时。会话状态通过Redis缓存管理，支持自动续期和单点登录。

2. 简历管理模块

支持多版本简历管理。学生可以使用已经设置好的文字简历模板来填写个人简历信息，也可以自主上传PDF简历，文字版的简历只需要填写必要信息，因为投递简历时可以将PDF简历作为附件一同投递，系统提供富文本编辑器，支持教育经历、工作经历、项目经验等结构化的数据输入，文字版简历数据采用JSON格式存储，便于前端渲染和后续的数据分析。学生求职者用户可存储多个版本的简历，在投递时可选择简历版本投递。

3. 经验分享社区模块

此模块设计参考了现代的社交平台的常见架构，支持内容创作、分类管理和互动交流。在文章内容发布和编辑功能上，采用Markdown富文本编辑器，支持图文混排、代码高亮、附件上传等高级格式；拥有保存草稿功能，防止内容丢失，支持多版本管理；系统还设置了多维度的标签系统，比如行业标签，岗位标签，公司标签等文章内容所相关的设置，方便读者查看和文章后续的推广和筛选；在文章详情页面，除了点赞收藏，还设置关注作者和举报作者功能，符合主流社交平台功能；下方还设置评论区，用户可以在合规情况下自由发言，评论的点赞和回复消息会同步到相关用户，在消息栏中出现提示，实现消息的实时通知。

4. 职位搜索与投递申请模块

在这个功能中，系统会推荐一些岗位，用户也能在搜索框输入关键词，系统通过字段匹配显示相关岗位，支持多条件组合查询，与经验社区相同，在搜索结果中可以查看岗位详情、收藏岗位或是可以选择直接建立聊天框联系企业，也可选择已经编辑好的相应简历进行投递，系统将提供投递状态跟踪功能，结合消息通知模块，投递状态更新时会将最新消息同步至消息栏。

5. 聊天室模块

此模块中，聊天室实现求职者用户与企业间的一对一联系的功能，有已读功能的设计，支持文件的发送，聊天消息会通过消息通知模块实时出现在通知栏，提高沟通效率。

6. 消息通知模块

采用WebSocket实现实时消息推送，支持多种通知类型，系统通知内容包括账户安全通知，平台公告等；业务通知包括学生端的简历投递进度反馈，面试邀请通知，文章点赞评论回复等消息提示，管理员端的用户举报消息提示，企业端的新简历投递申请信息通知和互动消息等。

4.2.2. 企业端功能模块设计

企业端模块为招聘企业提供全流程的人才选拔功能，采用的是微服务架构的设计，提升了可用性和扩展性。图4.3为企业端功能模块结构图。

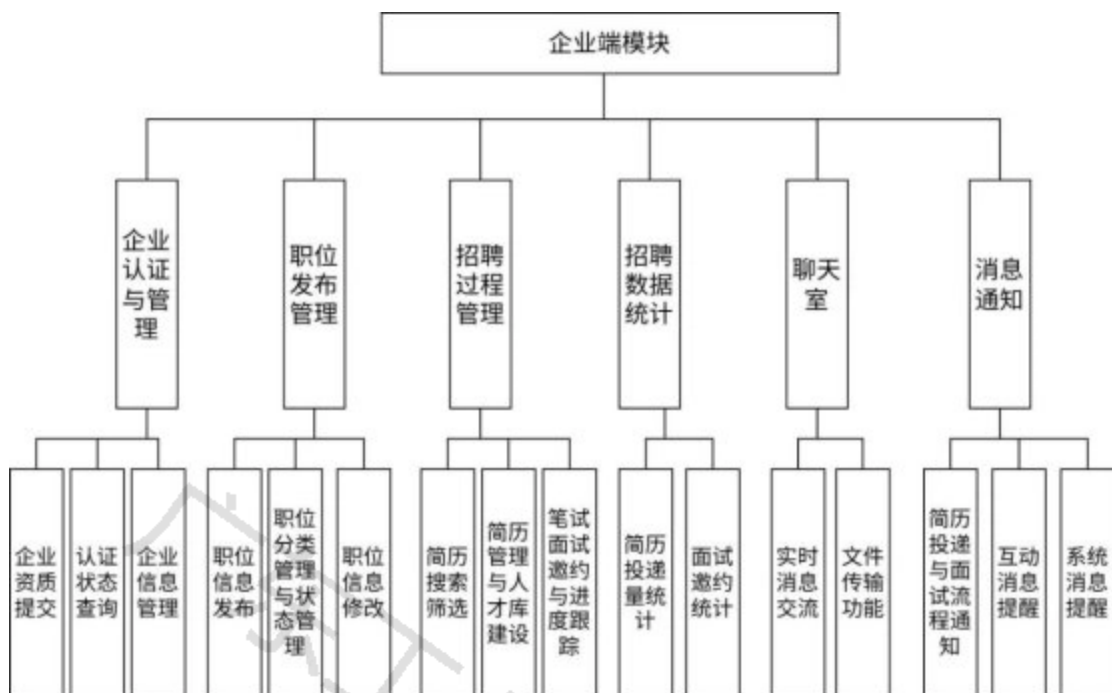


图4.3 企业端功能模块结构图

1. 企业认证与管理模块

该模块中企业用户注册时需要提供企业背景信息，企业运营资质，营业执照相关证书证明，按照普遍的企业认证审核需要的材料进行上传，上传之后，管理员端会接收到企业注册审核通知，在审核通过后企业才可进行后续的招聘操作。

2. 职位发布管理模块

系统为企业提供文字形式的招聘模板库，企业只需填写招聘需求等信息就可以快速发布职位招聘信息，企业同时也能够自由控制招聘信息的上架和删除，有草稿、已发布、已下架等状态。

3. 招聘过程管理模块

为企业端提供简历筛选功能，支持关键词筛选，如学校，学历，项目经历，是否有实习等；系统提供人才库创建功能，可将候选简历放入人才库中，方便招聘管理；结合消息通知系统，企业还可向求职学生发起笔面试邀请，并设置状态跟踪。

4. 招聘数据统计模块

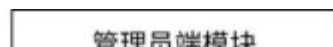
在企业个人主页页面设置招聘信息统计字段，进入主页即可直接查看统计信息，包括简历投递数、笔面试邀约数、录取数等详细招聘信息。

5. 聊天室模块

聊天室实现与求职者一对一联系的功能，有已读功能的设计，支持文件的发送，聊天消息会通过消息通知模块实时出现在通知栏，提高沟通效率。

4.2.3. 管理员端功能模块设计

管理员端采用的是Django内置的Django Admin管理员站点，它的防护和管理功能提高了系统的安全性和管理效率。图4.4为管理员端功能模块结构图。



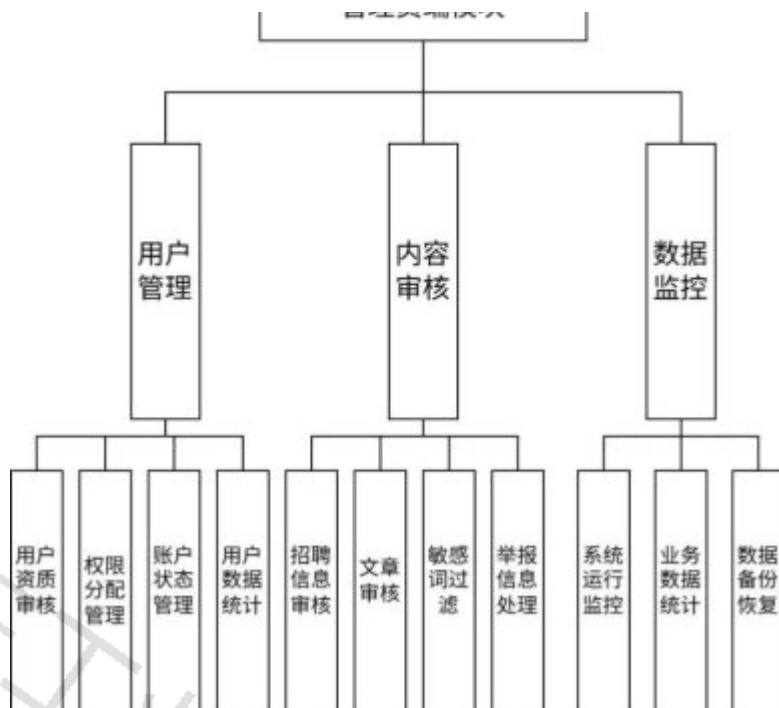


图4.4 管理员端功能模块结构图

1. 用户管理模块

用户管理模块实现对学生求职者用户和企业用户的管理，目前采用的是授权的形式，后续可扩展提升管理效率。

用户资质审核子模块目前采用一次性审批的设计，企业用户注册后，需要提交营业执照、组织机构代码证等资质文件，管理员通过识别，与第三方企业信息库进行交叉验证。学生用户则需要完成学籍认证，提供有效的官方学信网验证证明，后续可对接学信网API验证学历信息真实性，目前采用的是管理员手动验证。

权限管理子模块实现权限控制，本系统预定义了超级管理员，后续可以根据需要自主添加内容管理员和用户管理员等细分角色，管理员关联特定的操作权限和数据范围，权限分配支持批量操作。

用户行为监控机制放在通知模块中，会提示被举报的用户或者被举报的文章或评论所属用户，管理员可根据实际情况进行处置。

账户状态管理子模块提供灵活的状态变更功能，正常状态可以使用所有功能，冻结状态时用户账户暂时不可用，管理员可手动解冻，注销状态表示账户永久删除，相关数据一并删除或进行部分归档。

2. 内容审核模块

内容审核模块涵盖了招聘信息审核、经验帖审核、用户举报处理等核心功能，确保平台内容质量与合规性。

社区文章内容审核方面，管理员可设置对帖子标题、内容进行敏感词识别和关键词屏蔽功能，实现自动过滤和提醒修改，并且可以处理用户举报反馈；对广告内容进行识别下架，并对发布者进行提醒；评论区同样设置敏感词过滤，采用机器、人工结合方式，保证内容正向合规。

在企业招聘信息内容审核方面，重点对招聘内容和企业背景进行契合性检查，管理员可上下架企业招聘信息，并发送消息提醒；同样接受求职者对企业招聘信息的举报和反馈并进行处理回复。

3. 数据监控模块

本模块中，管理员通过后台管理系统查看系统运行状态，包括业务方面的：用户活跃度、业务数据（如文章发布量、举报量、简历投递量等）的统计；还有系统状态方面的（如数据吞吐量）的呈现；管理员可查看运行日志，必要时可以进行数据的备份，并且系统提供异常情况的提示和报警功能；

4.3. 系统数据库设计

4.3.1. 数据库概念模型设计

本系统采用关系型数据库MySQL作为主数据库，配合Redis作为缓存数据库。本平台的实体联系图如图4.5所示，以下为核心的实体及其关系：

1. 核心实体关系设计：

(1) 求职者用户与简历：一对多关系，一个求职者可以拥有多份简历，一个简历只属于一个求职者用户，通过用户ID作为外键在简历表中建立关联，支持简历的版本管理和默认设置；

(2) 求职者用户与职位申请：一对多关系，一个求职者可以投递申请多个职位，投递记录表通过用户ID和职位ID关联用户和职位实体；

(3) 企业与招聘信息：一对多关系，一个企业可以发布多条职位招聘信息，职位招聘信息表通过企业ID关联企业实体；

(4) 招聘信息与职位申请：一对多关系，一条招聘信息可以接收多个职位申请，职位申请表通过招聘信息ID关联招聘信息表；

(5) 简历与职位申请：一对多关系，一份简历可以有多个职位申请，职位申请通过简历ID关联简历；

(6) 企业与人才库：一对多关系，一个企业可以有多个人才库条目，人才库通过企业ID关联企业，在本平台，暂不作多个人才库的创建，默认一个企业对应一个人才库；

(7) 职位申请与人才库：一对一关系，由于本平台默认一个企业只有一个人才库，故一条职位申请只对应一个人才库，人才库通过职位申请ID关联职位申请表；

(8) 求职者用户与人才库：一对多关系，一位求职者用户可以存在于多个企业的人才库中，人才库通过求职者用户ID关联用户表；

(9) 求职者用户与文章：一对多关系，一个求职者可以发布多个文章，文章表通过用户ID关联用户表；

(10) 文章与评论：一对多关系，一篇文章可以有条评论；评论通过文章ID关联文章表；

(11) 求职者用户与评论：一对多关系，一个用户可以发表多个评论，评论表通过发表评论的用户ID关联用户表；

(12) 用户与聊天室：多对多关系，一个用户可以参与多个聊天室，一个聊天室有两个用户（企业和求职者），暂时没有群聊功能，聊天室通过双方用户ID关联用户表；

(13) 聊天室与聊天消息：一对多关系，一个聊天室可以有多个聊天消息，消息表通过聊天室ID关联聊天室表；

(14) 用户与通知：一对多关系，一个用户可以接收多条来信、互动消息和系统通知，每条通知只针对一个用户，通知消息表通过用户ID关联用户表；

(15) 管理员与用户：多对多关系，一个管理员可以管理多个用户，一个用户也可以被多个用户管理；

(16) 管理员与公告：一个管理员可以发布多条系统公告，每条公告只由一个管理员创建。公告表通过管理员ID

关联管理员用户表，此实体联系在ER图中不作呈现；

(17) 企业与认证信息：一对一关系，每个企业对应一份认证信息（如营业执照、企业资质），认证信息表通过企业ID与用户表关联，此联系存储在企业属性中，ER图不作呈现。

2. 实体属性设计：

(1) 用户实体：用户主键ID、用户名、密码、昵称、邮箱、手机号、用户类型、头像等属性；

(2) 企业实体：企业ID、企业名称、企业Logo、企业简介、所属行业、规模、联系人、联系方式、是否认证、外键为用户ID，关联用户表，企业实体是特殊的用户实体；

(3) 招聘信息实体：招聘信息主键ID、职位标题、工作类型、工作地点、薪资范围、招聘人数、经验要求、学历要求、外键为企业ID；

(4) 职位申请实体：职位申请ID、申请状态、申请时间、简历快照、外键为用户ID和招聘信息ID；

(5) 简历实体：简历主键ID、姓名、邮箱、电话、性别、教育经历、工作经验、项目经验、实习经历、技能、求职意向、PDF简历文件路径，外键为用户ID；

(6) 人才库实体：人才库主键ID、标签、招聘状态、外键为企业ID、用户ID和职位申请ID；

(7) 职位实体：职位主键ID、发布企业的用户ID、职位名称、职位详细描述、薪资范围、工作地点、职位类型（实习/全职）、职位状态（草稿、已发布）等属性；

(8) 文章实体：文章主键ID、标题、帖子内容、分类标签、浏览量、点赞数、收藏数、是否通过管理员审核、发布时间，外键为发布用户ID；

(9) 评论实体：评论主键ID、评论内容、评论时间、评论点赞数，外键为评论所属的文章ID和评论的用户ID和父评论ID（父评论ID自关联，用于评论回复逻辑）；

(10) 聊天室实体：聊天室主键ID、创建时间，有三个外键：企业用户ID、求职者用户ID和招聘信息ID；

(11) 消息实体：消息主键ID、消息类型、消息内容、是否已读、发送时间，外键为发送者ID和聊天室ID；

(12) 通知实体：通知主键ID、通知类型、通知标题、通知内容、是否已读、创建时间，外键为接收用户的ID。

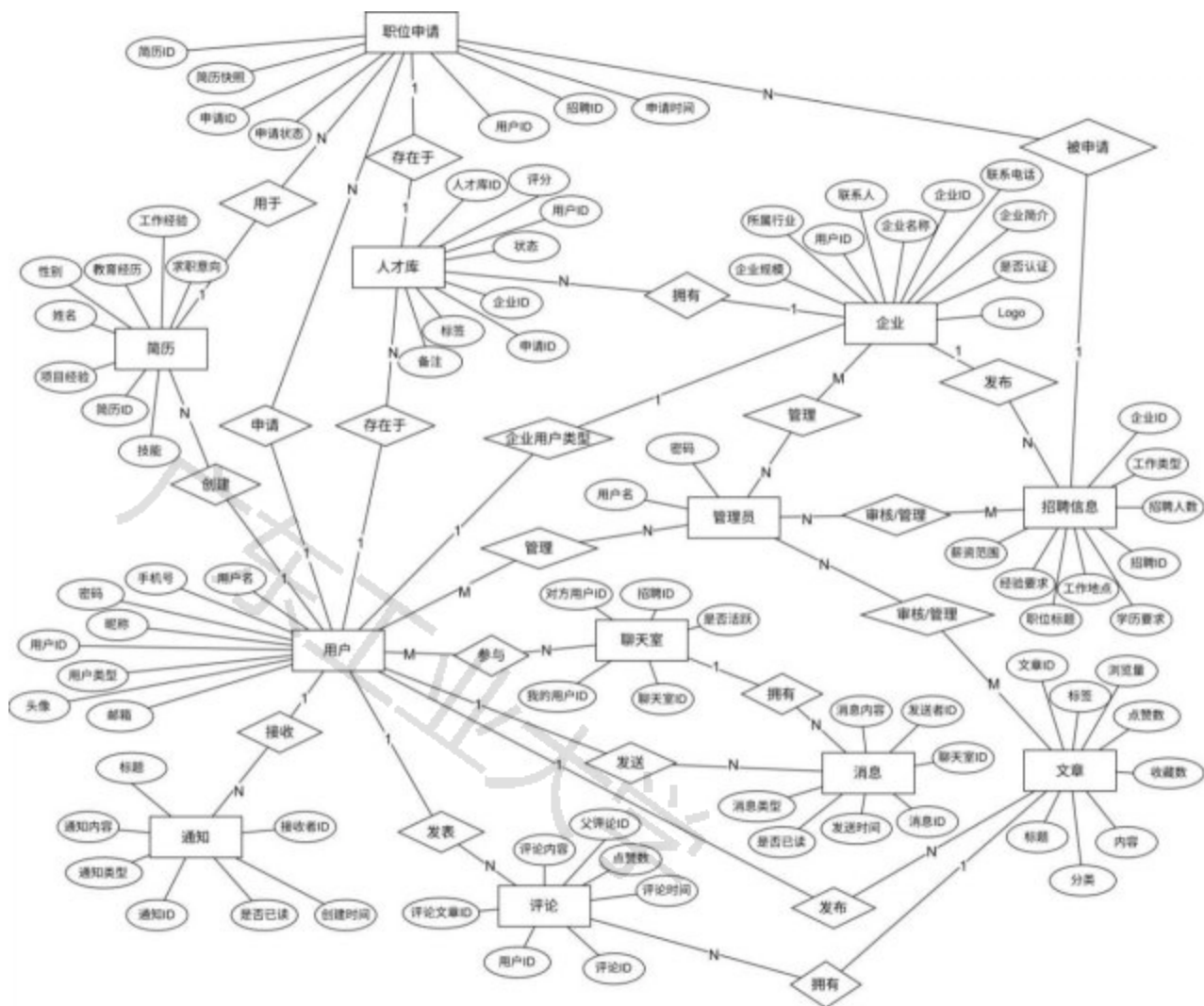


图4.5 大学生就业信息共享平台实体联系图（ER图）

上图4.5需要说明的是：为了避免ER图过于繁杂，此平台的数据库模型设计中，企业实体是用户实体的一个特例，即企业的用户信息也存储在总用户表中，用户表中设置了is_enterprise字段来区分用户是普通求职者用户还是企业用户，企业用户也拥有普通用户的功能模块，但是实体联系图（图4.5）中未对企业实体也进行重复的呈现，而是统一体现在“用户实体”上，比如：

1. 企业实体与聊天室的关系是一对多关系，一个企业可以参与多个聊天室，它在ER图中呈现为“用户与聊天室是多对多的关系，即一个用户可以参与加入多个聊天室，一个聊天室中可以有多个用户，一个是普通求职者用户，另一个可以是企业用户”；
2. 企业实体和消息实体也有一对多的关系：一个企业可以在聊天室内发送多个消息；
3. 企业实体也与通知实体有联系，即一对多关系，即一个企业用户可以接收多个通知；

简单来说，为了简化ER图，把普通用户实体和企业用户实体相同功能的实体关系统一由“用户”这个实体呈现，企业实体只呈现企业特有的属性和实体关系。

4.3.2. 数据库物理模型设计

大学生就业信息共享平台系统为每个微服务都设置独立的数据库，每个数据库都有多个数据表，以下是本平台核心的数据表物理模型结构的详细说明。

(1) 用户信息表如表4.1所示，用户信息表是系统的核心表，用于存储所有注册用户的基本信息，包括个人用户和企业用户的共同属性，如账号信息、个人资料等。系统通过该表区分用户类型，实现不同角色的功能访问控制。

表4.1 用户信息表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	用户ID
password	CharField(128)	NOT NULL	哈希加密的密码
last_login	DateTimeField	NULL	最后登录时间
is_superuser	BooleanField	NOT NULL	是否超级用户
username	CharField(150)	NOT NULL, UNIQUE	用户名
email	CharField(254)	NOT NULL	邮箱
is_staff	BooleanField	NOT NULL	是否管理员
is_active	BooleanField	NOT NULL	是否活跃
date_joined	DateTimeField	NOT NULL	注册时间
nickname	CharField(30)	NULL	用户昵称
sex	CharField(10)	NULL	性别
age	IntegerField	NULL	年龄
graduation_school	CharField(100)	NULL	毕业学校
education_level	CharField(50)	NULL	学历
major	CharField(100)	NULL	专业
graduation_year	IntegerField	NULL	毕业年份
phone_number	CharField(15)	NULL	电话号码
current_status	CharField(100)	NULL	当前状态
intended_position	CharField(100)	NULL	意向职位
intended_salary	CharField(50)	NULL	意向薪资
address	CharField(255)	NULL	地址
intended_city	CharField(100)	NULL	意向城市
personal_profile	TextField	NULL	个人简介
is_enterprise	BooleanField	NOT NULL	是否企业用户
avatar	ImageField	NULL	用户头像

特别说明：本系统通过Django内置的AbstractUser模型实现管理员角色，无需单独创建管理员表；自定义的用户信息表User继承自AbstractUser，其中：is_staff字段为True时，标记用户具有管理员身份、能够登录Django管理后台操作；is_superuser字段为True时，标记用户为超级管理员，拥有系统的所有权限，不过本系统不对is_staff和is_superuser作详细区分，默认所有管理员都拥有系统所有权限。is_active字段用于实现账号冻结/解

冻功能。当管理员将该字段设置为False 时，对应用户将无法登录系统，实现账号的临时禁用。

(2) 企业信息表如表4.2所示，企业信息表用来存储企业用户的详细信息，与用户表一对一关联。这个表记录了企业的基本资料、联系方式、行业规模等信息，并通过is_verified字段标记了企业是否通过平台认证。

表4.2 企业信息表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	企业ID
user_id	BigIntegerField	NOT NULL, UNIQUE	关联用户ID
name	CharField(200)	NOT NULL, UNIQUE	企业名称
logo	ImageField	NULL	企业Logo
description	TextField	NOT NULL	企业简介
industry	CharField(20)	NOT NULL	所属行业
scale	CharField(20)	NOT NULL	企业规模
contact_person	CharField(50)	NOT NULL	联系人
contact_phone	CharField(20)	NOT NULL	联系电话
contact_email	EmailFieldNOT	NULL	联系邮箱
address	CharField(500)	NOT NULL	企业地址
is_verified	BooleanField	NOT NULL	是否通过认证
created_at	DateTimeField	NOT NULL	创建时间
updated_at	DateTimeField	NOT NULL	更新时间

(3) 招聘信息表如表4.3所示，这个表用于存储企业发布的招聘岗位信息，包括岗位描述、任职要求、薪资待遇、工作地点等关键信息。它与企业表关联，能够让企业管理自己的招聘信息，并通过状态字段来控制招聘信息的发布状态。

表4.3 招聘信息表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	招聘 ID
enterprise_id	BigIntegerField	NOT NULL	所属企业 ID
title	CharField(200)	NOT NULL	招聘标题
job_type	CharField(20)	NOT NULL	工作类型
work_location	CharField(200)	NOT NULL	工作地点
salary	CharField(100)	NOT NULL	薪资范围
number_of_recruits	PositiveIntegerField	NOT NULL	招聘人数
experience	CharField(20)	NOT NULL	工作经验要求

education	CharField(20)	NOT NULL	学历要求
job	TextField	NOT NULL	职位名称
job_desc	TextField	NOT NULL	职位描述
job_require	TextField	NOT NULL	任职要求
status	CharField(20)	NOT NULL	招聘状态
deadline	DateField	NOT NULL	截止日期
created_at	DateTimeField	NOT NULL	创建时间
updated_at	DateTimeField	NOT NULL	更新时间

(4) 职位申请表如表4.4所示，这个表是求职者投递简历时生成的快照信息，用来存储投递信息，企业端能够看到投递进来的简历对应的岗位和状态信息。该表同时存储了投递时的简历快照，企业查看的是求职者投递时刻的简历版本。

表4.4 职位申请表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	申请ID
recruitment_id	BigIntegerField	NOT NULL	招聘信息ID
applicant_id	BigIntegerField	NOT NULL	申请用户ID
resume_id	BigIntegerField	NULL	简历ID
resume_snapshot	JSONField	NOT NULL	简历快照
pdf_file	FileField	NULL	PDF简历
status	CharField(20)	NOT NULL	申请状态
applied_at	DateTimeField	NOT NULL	申请时间

(5) 企业人才库表如表4.5所示，这个表用于企业存储和管理潜在人才信息，企业可以将优秀的求职者加入人才库进行再筛选和跟踪。该表与职位申请表关联，保留了求职者的简历快照等信息，方便企业进行人才筛选和管理。

表4.5 企业人才库表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	人才库ID
enterprise_id	BigIntegerField	NOT NULL	所属企业ID
job_seeker_id	BigIntegerField	NOT NULL	求职者ID
application_id	BigIntegerField	NULL	关联的申请记录ID
pdf_file	FileField	NULL	PDF简历
resume_snapshot	JSONField	NOT NULL	简历快照

tags	CharField(500)	NOT NULL	标签
status	CharField(20)	NOT NULL	人才状态
added_at	DateTimeField	NOT NULL	添加时间
updated_at	DateTimeField	NOT NULL	更新时间

(6) 简历表如表4.6所示，简历表用于存储求职者的个人简历信息，包括基本信息、教育背景、工作经验、项目经历等，求职者可以创建多个简历，方便在求职过程中灵活投递使用。

表4.6 简历表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	简历ID
user_id	BigIntegerField	NOT NULL	关联用户ID
name	CharField(50)	NOT NULL	姓名
sex	CharField(10)	NOT NULL	性别
email	EmailField	NOT NULL	邮箱
phone	CharField(20)	NOT NULL	电话
education	CharField(50)	NOT NULL	学历
internship_experience	TextField	NOT NULL	实习经历
work_experience	TextField	NOT NULL	工作经历
project_experience	TextField	NOT NULL	项目经历
school_experience	TextField	NOT NULL	校园经历
self_evaluation	TextField	NOT NULL	自我评价
scholarships	TextField	NOT NULL	奖学金
skills	CharField(500)	NOT NULL	技能标签
job_objective	CharField(100)	NOT NULL	求职意向
status	CharField(20)	NOT NULL	状态
pdf_url	FileField	NULL	PDF的存储地址
created_at	DateTimeField	NOT NULL	创建时间
updated_at	DateTimeField	NOT NULL	更新时间

(7) 文章表如表4.7所示，这个表存储用户发布的各类职场相关文章，包括面试经验、简历技巧、行业分析、吐槽推荐等内容，该表支持文章分类并记录文章的收藏数、点赞数等互动数据。

表4.7 文章表

字段名	数据类型	约束	描述

id	BigAutoField	PRIMARY KEY	文章ID
user_id	BigIntegerField	NOT NULL	作者ID
title	CharField(80)	NOT NULL	标题
category	CharField(20)	NOT NULL	分类
tags	CharField(50)	NOT NULL	标签
content	TextField	NOT NULL	内容
view_count	PositiveIntegerField	NOT NULL	浏览量
like_count	IntegerField	NOT NULL	点赞数
star_count	IntegerField	NOT NULL	收藏数
comment_count	IntegerField	NOT NULL	评论数
created_at	DateTimeField	NOT NULL	创建时间
updated_at	DateTimeField	NOT NULL	更新时间

(8) 评论表如表4.8所示，这个表用来存储用户在文章下的评论，支持评论的嵌套回复功能。该表记录了评论者信息、评论内容和评论时间等。

表4.8 评论表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	评论ID
article_id	BigIntegerField	NOT NULL	所属文章ID
user_id	BigIntegerField	NOT NULL	评论者ID
parent_id	BigIntegerField	NULL	父评论ID
content	TextField	NOT NULL	评论内容
created_at	DateTimeField	NOT NULL	评论时间
like_count	PositiveIntegerField	NOT NULL	点赞数

(9) 聊天室表如表4.9所示，用于创建和管理企业与求职者之间的聊天会话，该表记录了聊天双方的信息、关联的招聘岗位。

表4.9 聊天室表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	聊天室ID
enterprise_user_id	BigIntegerField	NOT NULL	企业用户ID
job_seeker_user_id	BigIntegerField	NOT NULL	求职者用户ID
recruitment_id	BigIntegerField	NULL	关联招聘信息ID

is_active	BooleanField	NOT NULL	是否活跃
created_at	DateTimeField	NOT NULL	创建时间
updated_at	DateTimeField	NOT NULL	最后活动时间

(10) 聊天消息表如表4.10所示，该表用来存储聊天室内的消息记录，支持文本消息和文件消息。而且记录了消息发送者、内容、类型和发送时间等信息，并可以通过已读状态字段追踪消息的阅读情况。

表4.10 聊天信息表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	消息ID
chat_room_id	BigIntegerField	NOT NULL	聊天室ID
sender_id	BigIntegerField	NOT NULL	发送者ID
content	TextField	NOT NULL	消息内容
message_type	CharField(20)	NOT NULL	消息类型
file	FileField	NULL	文件
file_name	CharField(255)	NOT NULL	文件名
file_size	IntegerField	NOT NULL	文件大小
is_read	BooleanField	NOT NULL	是否已读
read_at	DateTimeField	NULL	阅读时间
created_at	DateTimeField	NOT NULL	发送时间

(11) 通知信息表如表4.11所示，通知信息表用于存储系统向用户发送的各类通知信息，包括简历投递、面试邀请、消息提醒等。支持不同类型的通知，并通过已读状态字段管理通知的阅读状态，确保用户及时获取重要信息。

表4.11 通知信息表

字段名	数据类型	约束	描述
id	BigAutoField	PRIMARY KEY	通知ID
recipient_id	BigIntegerField	NOT NULL	接收者ID
notification_type	CharField(50)	NOT NULL	通知类型
title	CharField(200)	NOT NULL	通知标题
message	TextField	NOT NULL	通知内容
is_read	BooleanField	NOT NULL	是否已读
related_object_id	PositiveIntegerField	NULL	关联对象ID
related_object_type	CharField(50)	NULL	关联对象类型
created_at	DateTimeField	NOT NULL	创建时间

4.4. 本章小结

本章完成了大学生就业信息共享平台的整体架构设计，功能模块的划分以及数据库的设计。架构使用前后端分离架构，通过Vue.js构建前端交互界面，Django REST Framework提供RESTful API接口，结合MySQL主数据库与Redis缓存数据库的混合存储策略，以实现高内聚低耦合的设计目标；功能模块完成了学生求职者用户端、企业用户端、管理员端的更为精细的功能划分，展示了核心功能模块和后续详细设计实现方向；数据库设计按照系统的业务逻辑完成了实体关系建模与物理模型设计，并且使用ER图展示了各实体之间的核心关系，设计了11张核心数据表，保证数据存储的完整性与一致性，适配各功能模块的数据交互需求。

5. 系统详细设计与实现

5.1. 系统技术分析

本系统采用前后端分离的架构设计，前端使用Vue.js框架构建用户交互界面，后端基于Django REST Framework提供RESTful API接口，数据库采用MySQL关系型数据库存储核心业务数据，Redis缓存数据库用于提升系统性能和实现实时消息推送。

前端部分采用Vue3框架，配合Naive UI组件库实现具有现代美感而又简约的用户界面，前端技术栈包括：

1. Vue 3: 采用Composition API编写组件，提供更好的代码组织和复用性
2. Vue Router: 实现单页面应用的路由管理
3. Pinia: 状态管理工具，管理全局状态如用户信息、聊天消息等
4. Axios: HTTP客户端，用于与后端API进行数据交互
5. WebSocket: 实现实时消息推送和聊天功能

后端采用Django框架，基于Django REST Framework构建RESTful API，后端技术栈包括：

1. Django: Web框架，提供MTV架构模式
2. Django REST Framework: 提供序列化、认证、权限等功能
3. JWT认证: 基于JSON Web Token的用户认证机制
4. Celery: 异步任务队列，用于发送邮件、处理耗时任务
5. Redis: 缓存数据库，用于会话管理和实时消息推送

5.2. 系统功能模块详细设计与实现

5.2.1. 用户注册登录模块

此模块是这个系统运行的基础，也是开始，采用Django的认证系统和自定义用户模型来实现，实现的是普通求职者用户和企业用户的注册、登录以及密码重置功能。

本模块详细设计思路流程如下：

1. 自定义用户模型，继承Django的AbstractUser模型，在User模型中添加了求职者所需字段，如昵称、性别、年龄、毕业学校、学历、专业、意向职位、意向薪资等；同时，通过is_enterprise布尔字段来区分用户模型（求职者用户和企业用户）。
2. 注册流程实现：注册时，前端发送请求到/api/register/接口，后端的register视图函数接收数据后使用

RegisterSerializer进行数据验证，它会验证用户名、邮箱的唯一性，以及两次密码是否一致。通过后，调用User.objects.create_user()方法来创建用户，密码也会加密存储。

3. 密码重置流程：当用户忘记密码时，系统提供一个安全重置机制，用户先通过send_reset_code接口获取验证码，后端生成4位随机验证码并存入Redis缓存，设置5分钟有效期，然后用户提交验证码和新密码，后端验证成功后，使用user.set_password()方法更新密码。

用户注册登录模块类图如图5. 1所示。

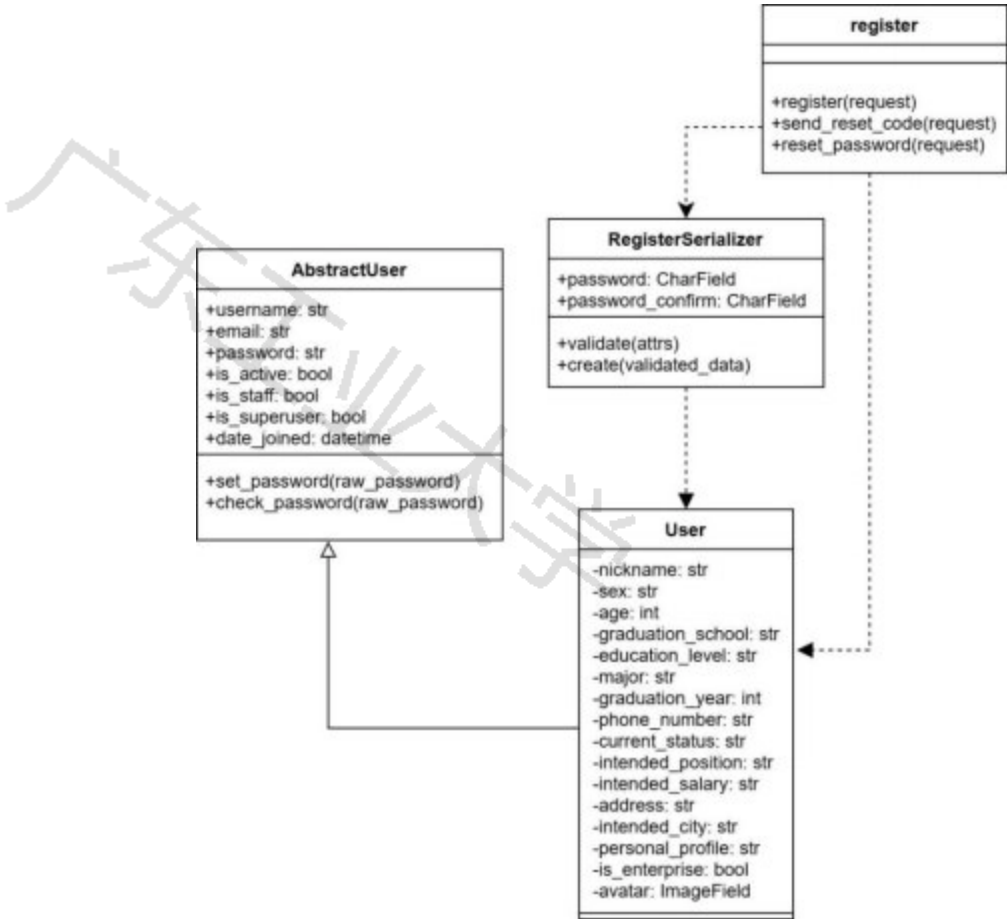


图5. 1 用户注册登录模块类图

用户注册时序图如图5. 2所示。

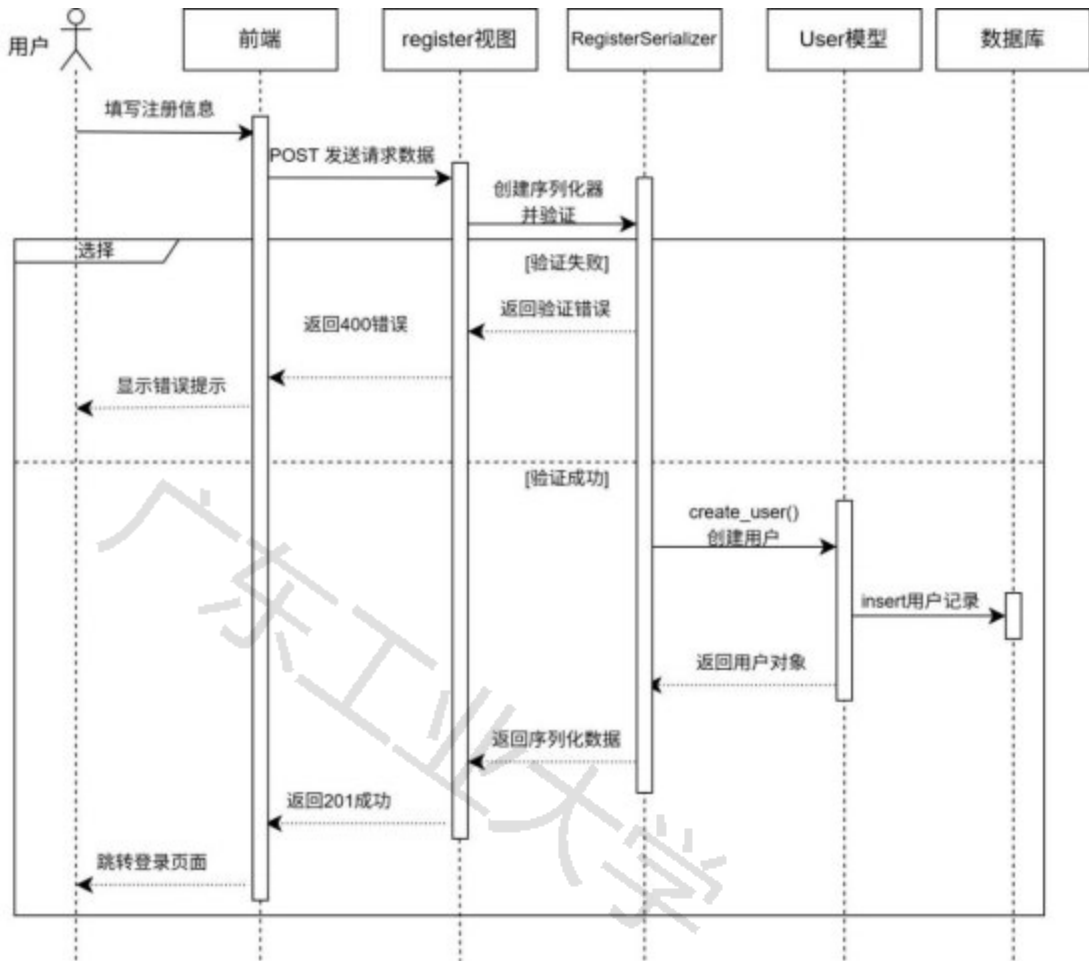


图5.2 用户注册时序图

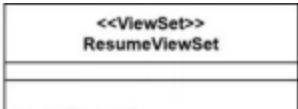
5.2.2. 简历管理模块

简历管理模块是求职者用户的核心功能之一，本系统使用Django REST Framework的ViewSet来实现完整的CRUD操作，并支持PDF文件上传。

本模块详细设计思路如下：

1. 简历模型Resume中：设计了完整简历字段，包括基本信息和各种普遍的简历必须字段，模型定义的具体字段在总体设计章节部分有展示，这里不多赘述，需要说明的是，Resume模型通过status字段区分草稿和已发布状态。
2. 权限控制方面：创建自定义权限类IsResumeOwner，确保用户只能访问和修改自己的简历。在get_queryset()方法中，通过Resume.objects.filter过滤出当前用户的简历。
3. 由于求职过程中通常需要上传PDF简历，因此在create方法中，也支持文件的上传，首先保存文字简历，然后检查请求中是否有pdf_file文件，如果有，将其保存到pdf_url字段。在update方法中，如果上传了新的PDF简历文件，就会删除旧文件，再保存新文件。
4. 在序列化器设计上，ResumeSerializer不仅处理基本字段，还添加了get_pdf_url()方法来生成完整的PDF文件URL，以及validate_skills()方法来处理技能标签的去重和格式化。

简历管理模块类图如图5.3所示，创建简历时序图如图5.4所示。



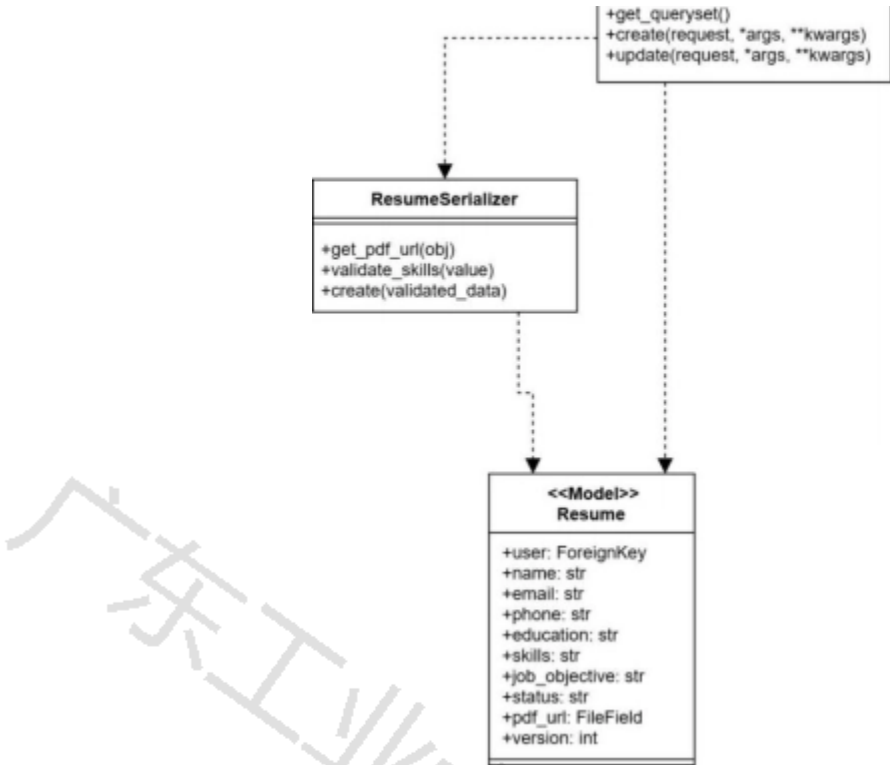


图5.3 简历管理模块类图

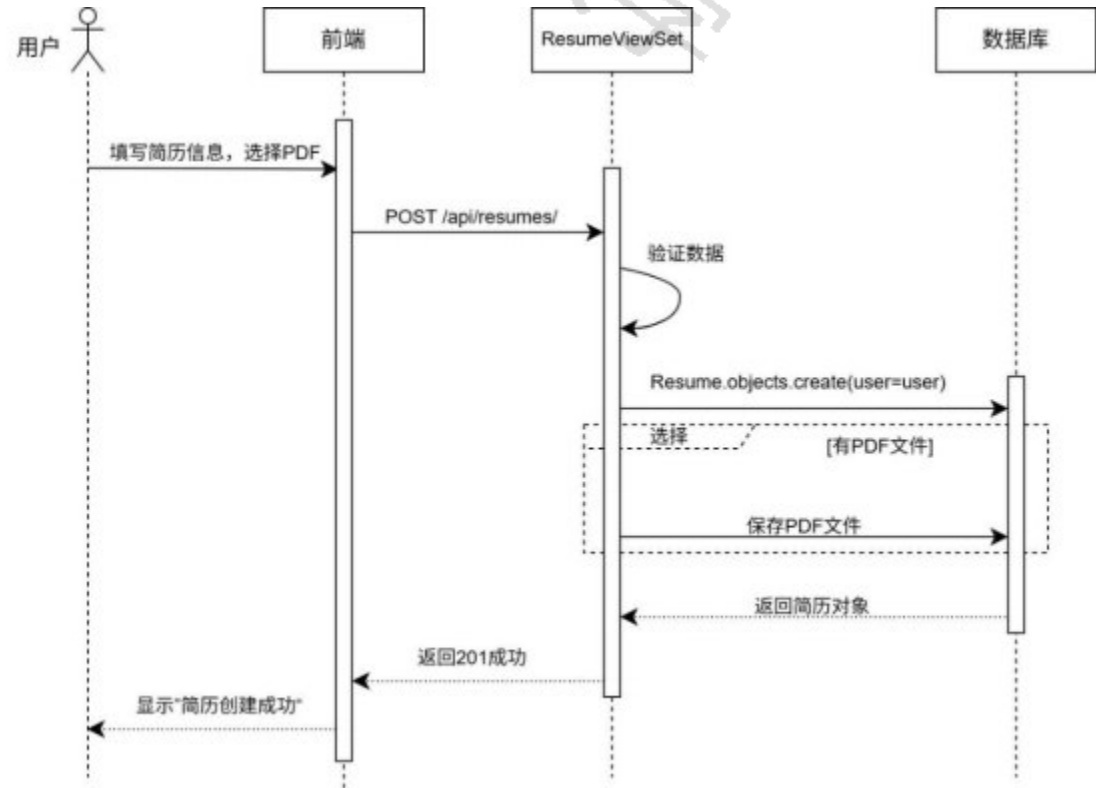


图5.4 创建简历时序图

5.2.3. 经验社区模块的详细设计与实现

经验社区模块是本系统的重要组成部分，实现文章发布、文章展示、评论点赞收藏和关注等完整的社交功能。

本模块详细设计思路如下：

- 1. 文章Article模型中，设计包括标题、分类、标签、内容以及互动数据字段（点赞评论收藏）。分类包括面试经验、简历技巧、行业选择、笔试攻略等。
 - 2. 评论系统的Comment模型支持多级评论，通过parent字段实现评论回复功能，顶级评论的parent为null，回复评论的parent指向被回复的评论。
 - 3. 点赞收藏功能中，通过Like模型和Collection模型实现点赞和收藏功能。这两个模型都使用unique_together约束以防止用户重复点赞或收藏同一文章。
 - 4. 在ArticleViewSet中，实现动态权限控制。查看文章允许所有用户（包括未登录），点赞和收藏需要登录，修改和删除文章需要是文章作者。
 - 5. 当用户点赞、收藏、评论文章时，系统会通过create_notification工具函数发送实时通知给文章作者。
- 经验社区模块类图如图5.5所示，发布文章时序图如图5.6所示，发表评论时序图如图5.7所示。

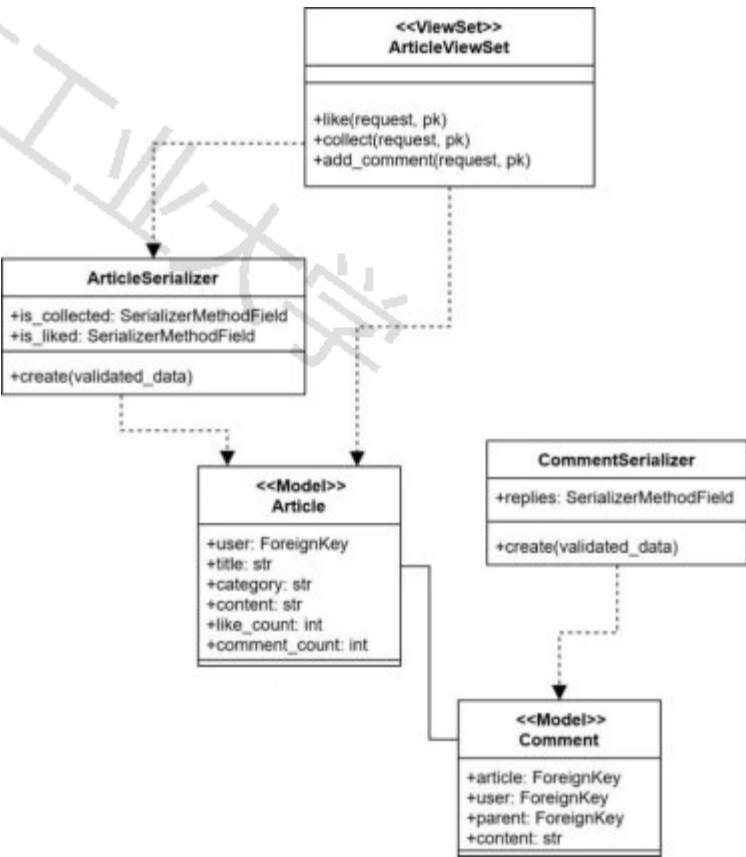
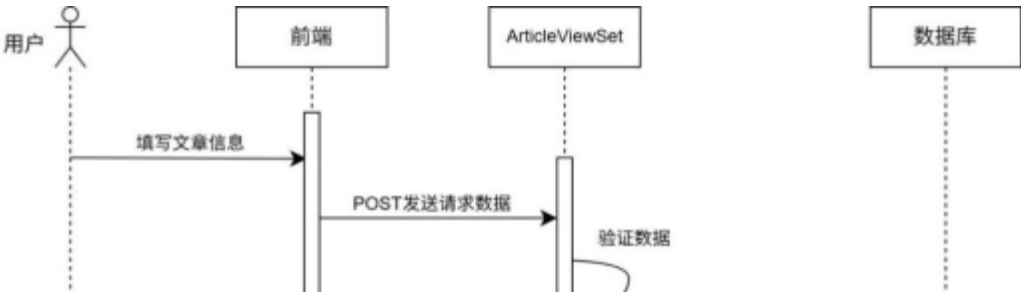


图5.5 经验社区模块类图

在经验社区模块中还有子模型，如存储点赞收藏关注的模型类，图5.5没有尽数画出，图中只呈现了最主要的类和关系。



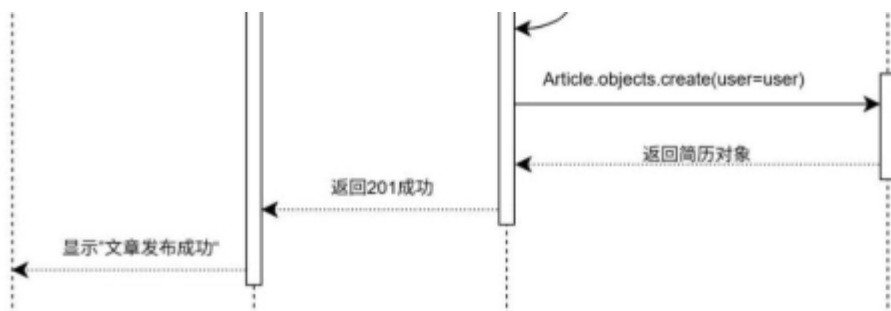


图5.6 发布文章功能时序图

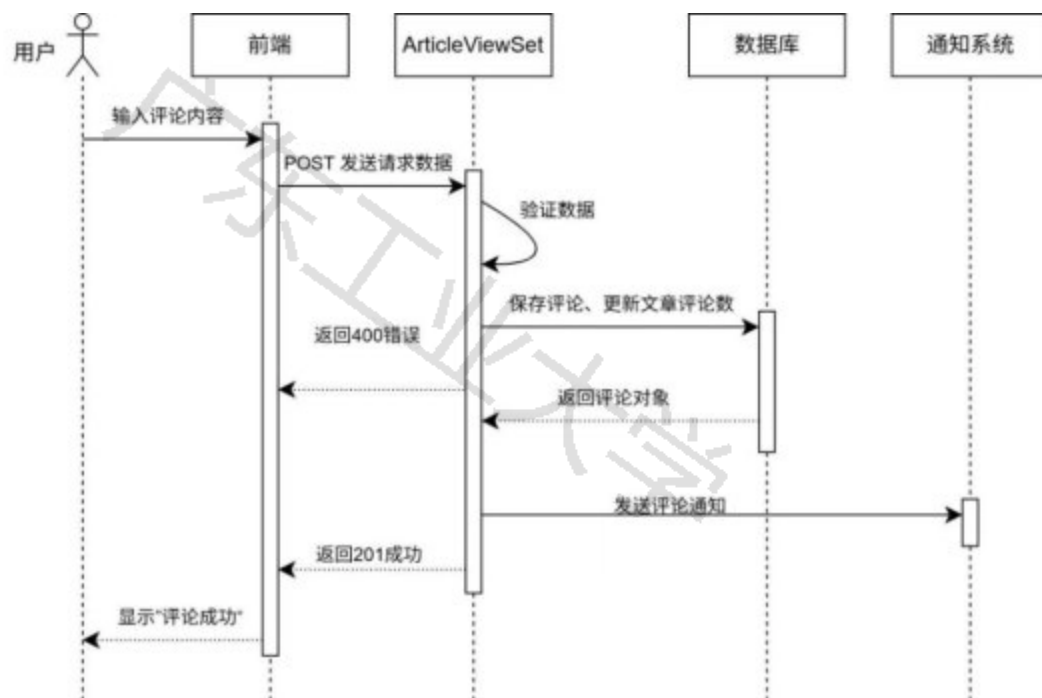


图5.7 发表评论功能时序图

5.2.4. 职位筛选和简历投递申请模块

本模块详细设计思路如下：

1. 职位筛选功能：求职者在前端页面选择筛选条件（如工作地点、学历要求、职位类别等）后，前端发送GET请求到/api/recruitments/ 接口，并将筛选的参数作为查询字符串传递；后端的get_queryset()方法会先判断用户的类型，若是企业用户，则查询自己的所有招聘信息，如果是普通求职者用户，则查询status为已发布的招聘信息，然后应用前端传的筛选条件过滤，最后按最后招聘信息的创建时间倒序排序再呈现到页面。

2. 简历详情查看功能：当求职者用户点击招聘信息卡片时，前端发送GET请求到/api/recruitments/{id}/接口，这里的id是职位信息的唯一标识，后端的 RecruitmentViewSet的retrieve()方法会被调用（继承自ModelViewSet），获取指定id的招聘信息对象，然后使用序列化器进行序列化，这个序列化器会自动嵌套企业信息，包括企业名称、Logo、行业等详细信息，通过get_enterprise_logo()方法生成完整的Logo URL，最后返回完整的职位详情数据给前端，求职者可以查看职位描述、任职要求、薪资待遇、工作地点等所有信息。

3. 选择简历并投递功能：求职者在职位详情页选定简历发起投递后，前端携带职位ID 与简历ID等参数向/api/applications/接口发送POST请求，后端随即调用 JobApplicationViewSet的create()方法，先通过权限类校验用

户求职者身份，再执行snapshot()方法提取姓名、学历技能等关键信息生成JSON简历快照，然后复制简历PDF文件并绑定申请记录防止原文件丢失，随后创建关联招聘岗位、求职者与简历快照的JobApplication申请数据，在这些完成后，调用通知函数向企业推送新简历提醒，实现简历投递全流程。

职位筛选功能时序图如图5.8所示，选择简历并投递功能时序图如图5.9所示。

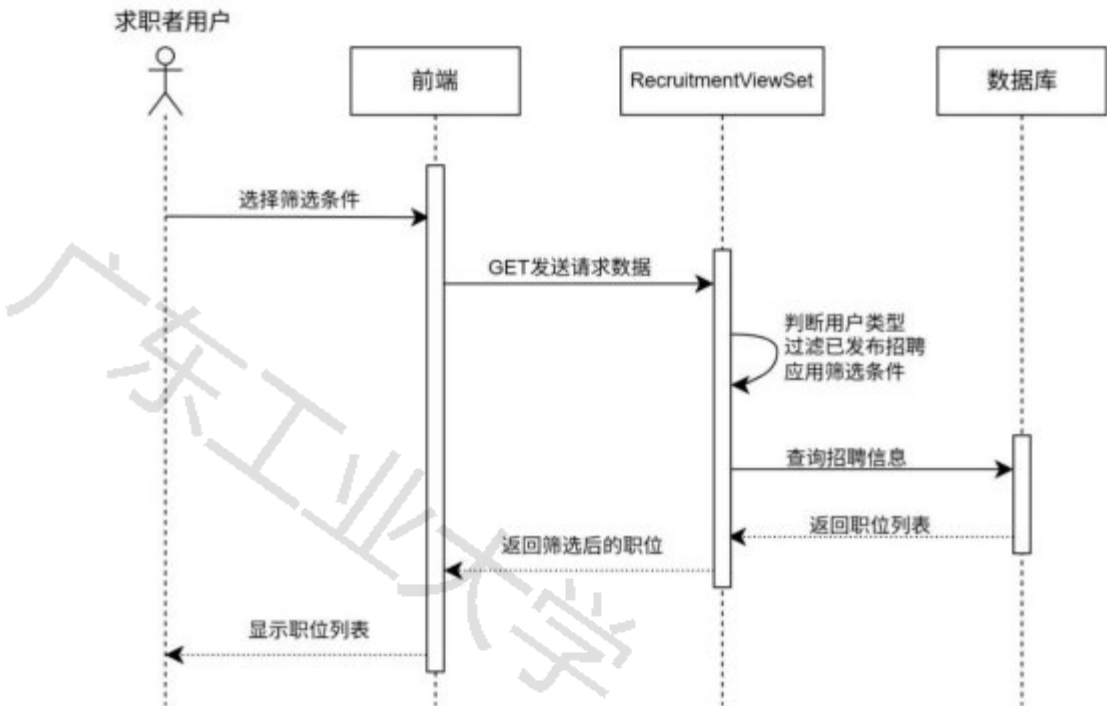


图5.8 职位筛选功能时序图

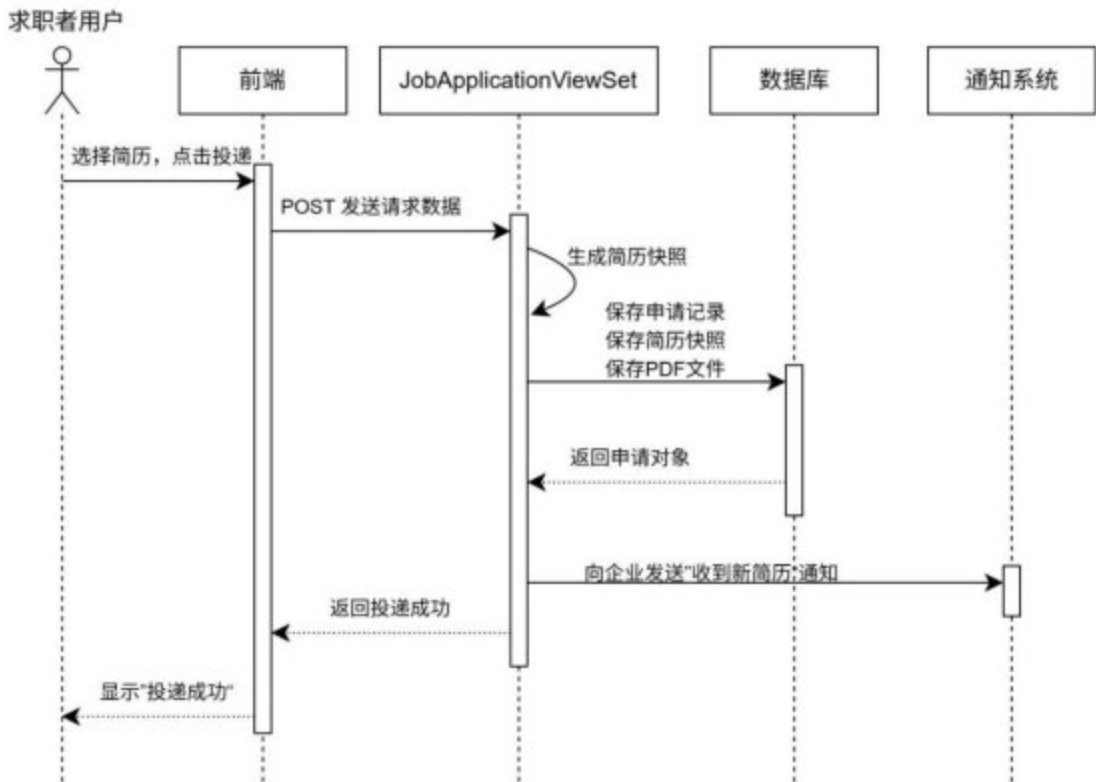


图5.9 选择简历并投递功能时序图

5.2.5. 聊天室模块

本模块详细设计思路如下：

- 1. 实时聊天室功能：当用户打开聊天界面时，前端会带着有聊天室唯一ID的标识建立WebSocket连接，后端的ChatConsumer的connect()方法会获取验证登录信息，通过的情况下，就把用户加到对应的聊天室并接通连接；用户发消息时，前端会通过WebSocket传输JSON格式的信息，后端的receive()方法接收消息，按消息类型分类处理。handle_chat_message()方法验证用户是否已认证，调用save_message()方法将消息保存到数据库，这个方法使用装饰器将同步数据库操作转换为异步操作。保存成功后通过channel_layer.group_send()将消息广播给聊天室内的所有用户，实现实时消息推送。
 - 2. 文件传输功能：用户选择文件后，前端通过Websocket发送消息，消息类型为file，内容为文件的名称和URL，后端ChatConsumer的handle_chat_message()方法接收消息，调用save_message()保存，message_type为file时，content存储文件名，并将文件URL保存到file字段里，保存成功后通过channel_layer.group_send()将文件消息广播给聊天室内的用户，接收方可以点击下载查看文件。
- 聊天室模块类图如图5.10所示，实时消息交流功能时序图如图5.11所示。

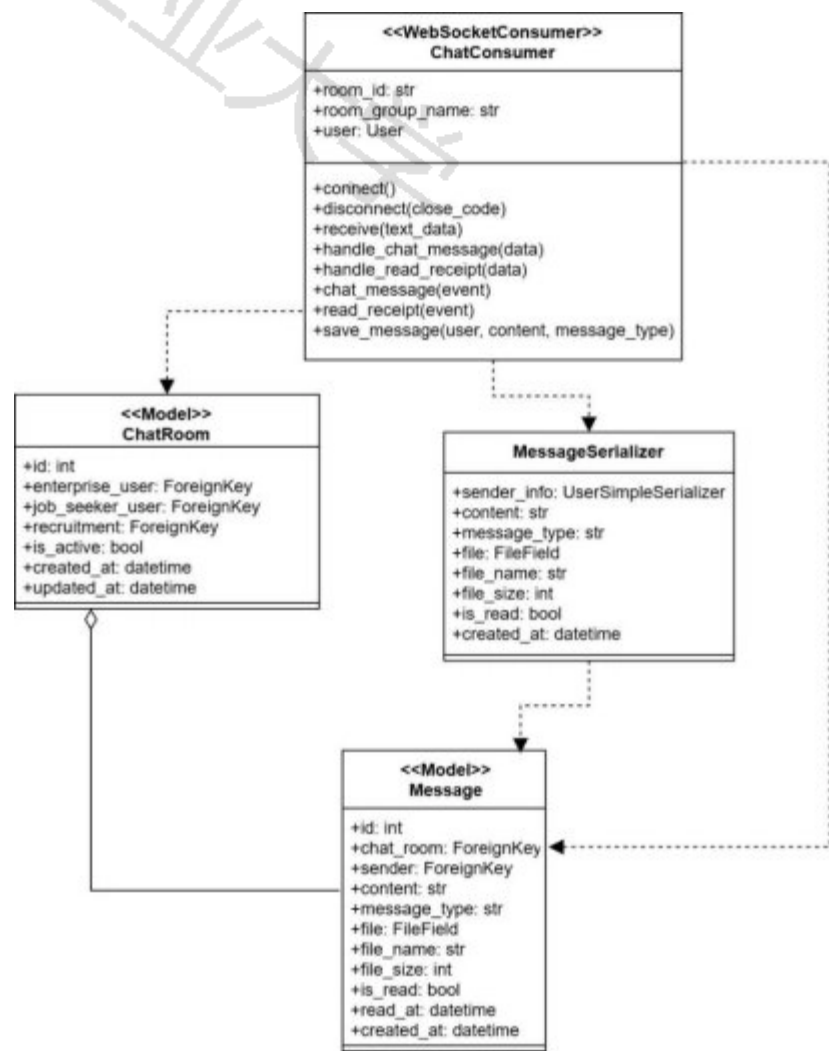


图5.10 聊天室模块类图

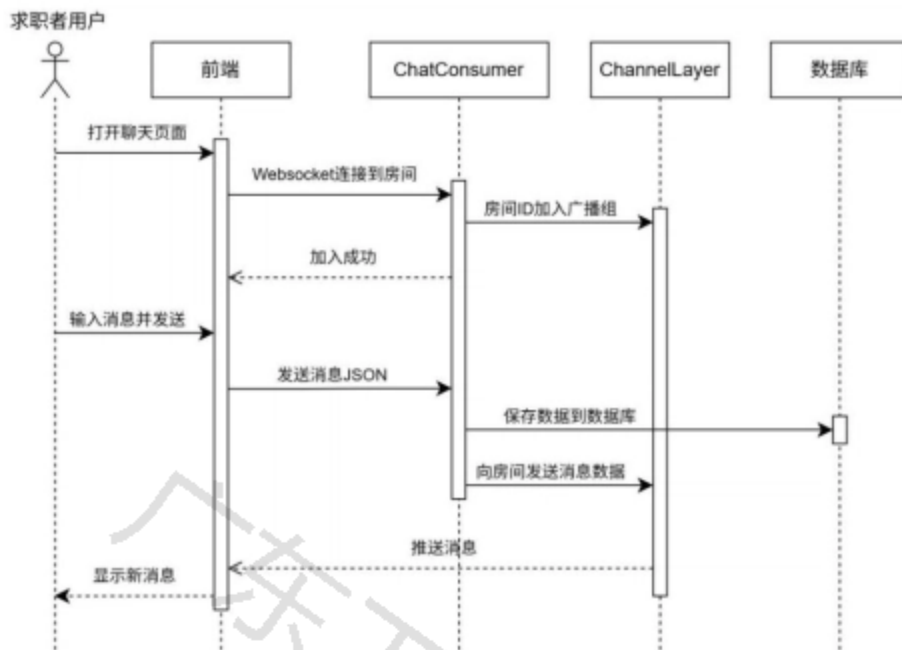


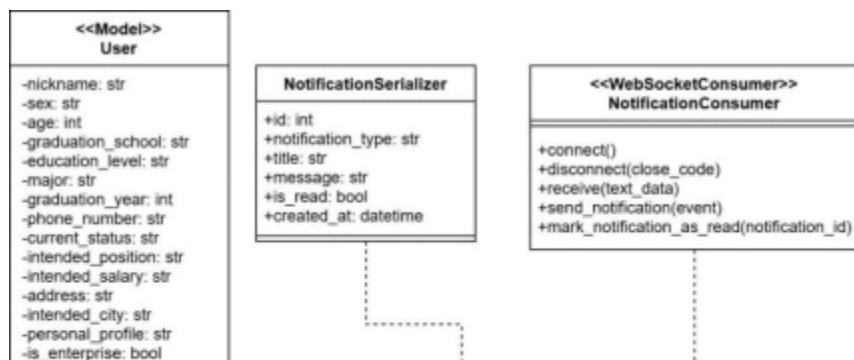
图5.11 实时消息交流功能时序图

5.2.6. 消息通知模块

本模块详细设计思路如下：

1. 招聘进度通知：企业用户修改求职者的申请状态后，前端会往update_status/接口发带新状态的POST请求，后端核对用户身份之后更新申请记录。之后用create_notification生成对应的通知（这个通知会关联对应的申请），先把通知存进数据库，再用send_notification_via_websocket()借助WebSocket推给求职者，让对方实时收到状态变更提醒。
2. 互动消息提醒：用户给文章点赞、收藏或者评论时，后端会自动触发通知。比如点赞时，后端的ArticleViewSet里的like()方法会启动，先检查用户有没有点过赞，没点过就创建点赞记录、更新文章点赞数，接着调用create_notification函数创建通知。之后把通知存进数据库，再用WebSocket实时推送给文章作者，作者就能立马收到互动提醒，收藏和评论的流程和与点赞时基本一致，也会触发对应的通知。
3. 系统通知提醒：系统要给用户发重要通知时，比如平台公告、审核反馈等，管理员或系统直接调用create_notification函数创建通知，标题和内容系统预设，管理员也能自行修改，不用关联具体的对象，这个函数把通知存进数据库后，再WebSocket实时推给目标用户，用户无需刷新页面就能接收。

图5.12为消息通知模块类图，图5.13为互动消息通知时序图，由于消息通知功能的设计思路和函数相似，为避免重复和节约篇幅，只给出互动消息通知的时序图。



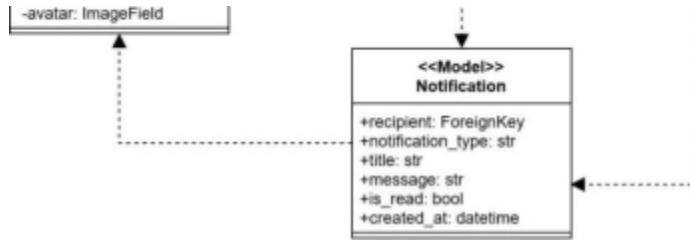


图5.12 消息通知模块类图

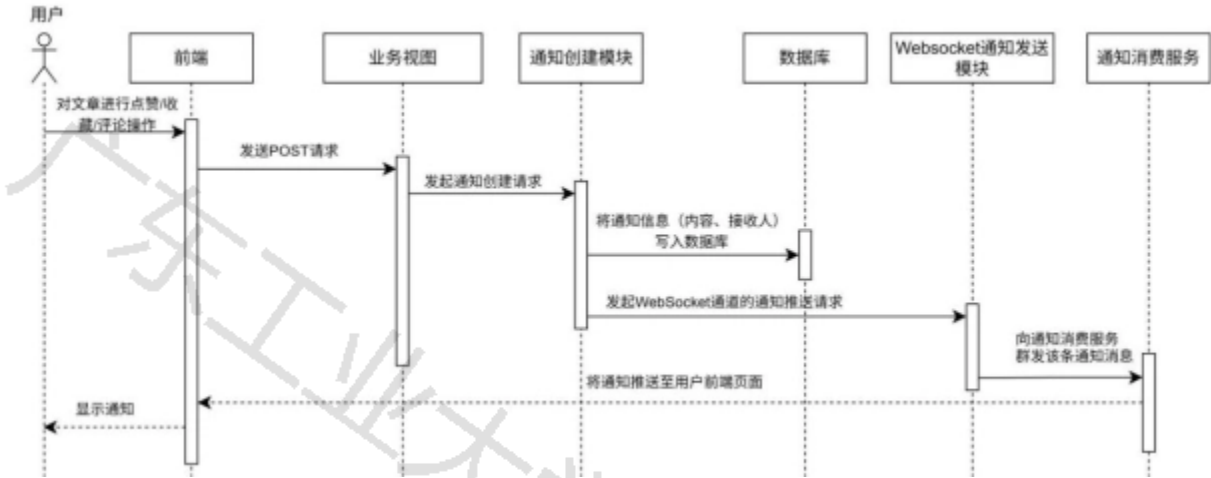


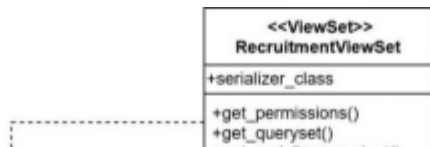
图5.13 互动消息通知时序图

5.2.7. 企业端职位发布与管理模块

本模块详细设计思路如下：

1. 职位信息发布：企业填好各类招聘信息后，前端随后发POST请求到指定接口，还携带是否立即发布的参数，后端触发RecruitmentViewSet的创建逻辑，验证企业用户身份后调用perform_create()自动关联当前用户的企业信息，再根据是否需要立即发布来把职位状态设为PUBLISHED（已发布）或DRAFT（草稿），最后把数据存到数据库，返回创建好的招聘数据。
2. 招聘信息分类与状态管理：企业在招聘列表里可以调整招聘信息的分类和状态，前端发送请求到/api/recruitments/{id}/，携带分类字段和状态参数，后端调用RecruitmentViewSet的update()方法，先通过get_object()获取对应ID的招聘信息，验证用户是否为企业所属企业，再用RecruitmentSerializer的update()方法处理更新，更新完返回数据。企业还能设deadline，也就是招聘截止日期，如果过期，那么这个招聘信息就下架。
3. 招聘信息修改功能基本流程也是先获取对应ID的招聘信息，再验证身份，接着用RecruitmentSerializer做数据验证和更新。

图5.14为企业端职位发布与管理模块类图，图5.15为职位信息发布时序图，图5.16为招聘信息分类与状态管理时序图。



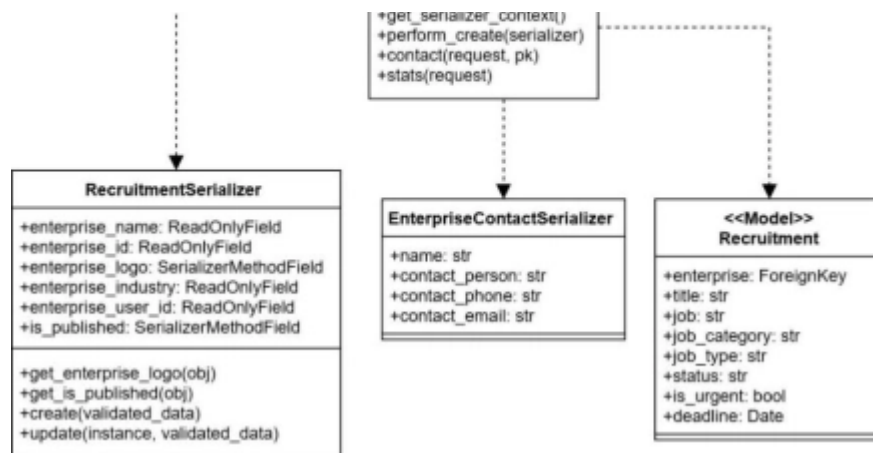


图5.14 企业端职位发布与管理模块类图

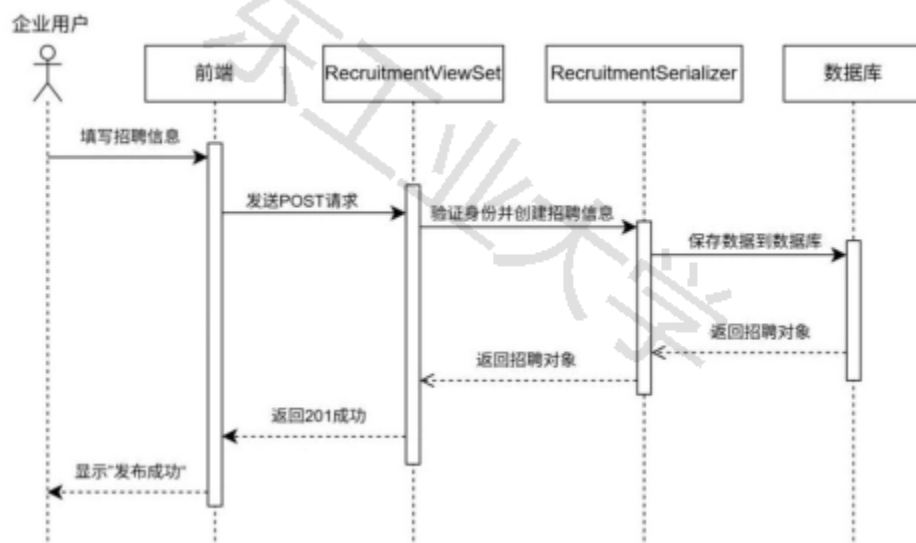


图5.15 职位信息发布时序图

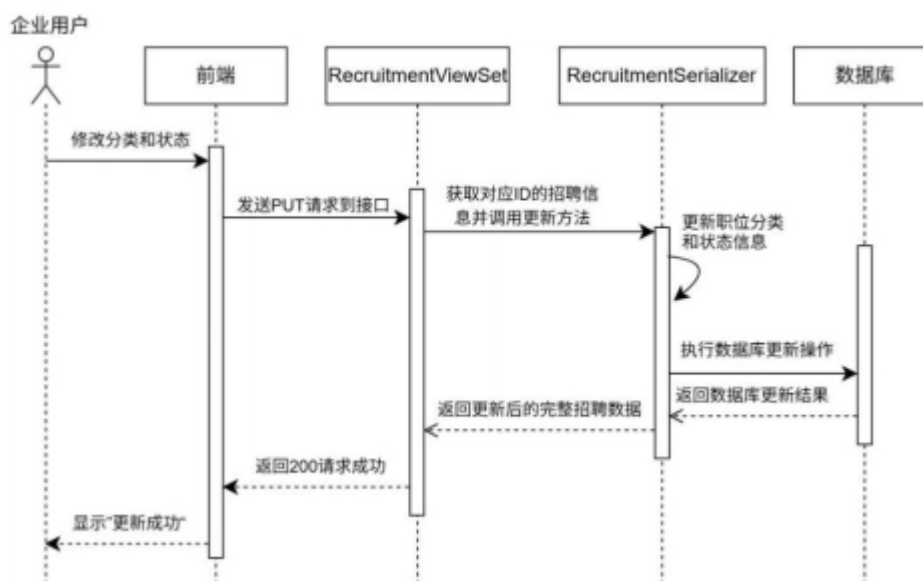


图5.16 招聘信息分类与状态管理时序图

5.2.8. 企业端招聘过程管理模块

本模块详细设计思路如下：

- 1. 简历管理与人才库建设：企业在进入简历管理页面时，前端发GET请求，后端视图JobApplicationViewSet和之前的描述一样，判断身份再查询当前企业的所有简历申请，这里还使用select_related() 预加载关联信息提高查询效率。在简历列表可以点 “加入人才库”，后端TalentPoolViewSet视图的方法会先验证申请是否存在、是否属于当前企业，再检查求职者是否已在人才库；若不在，就创建TalentPool对象，关联求职者、申请记录、简历快照和PDF文件，后续企业能在人才库进行下一环节的筛选，可以发送笔面试通知、发起聊天、查看详细简历信息等。
- 2. 笔试面试邀约与进度跟踪：企业在人才库页面可以更新申请状态，点击更新状态后，前端会发请求到接口，携带新状态（如初筛、待笔面试等）；后端调用JobApplicationViewSet的update_status() 方法，更新status字段并存入数据库。更新成功后点“发送通知”，调用create_notification函数给求职者发实时通知。这个模块还提供批量更新申请状态的功能，后端调用bulk_update_status() 方法批量更新提升效率。

图5.17为企业端招聘过程管理模块类图，图5.18为人才库建设功能时序图，图5.19为笔试面试邀约与进度跟踪功能时序图。

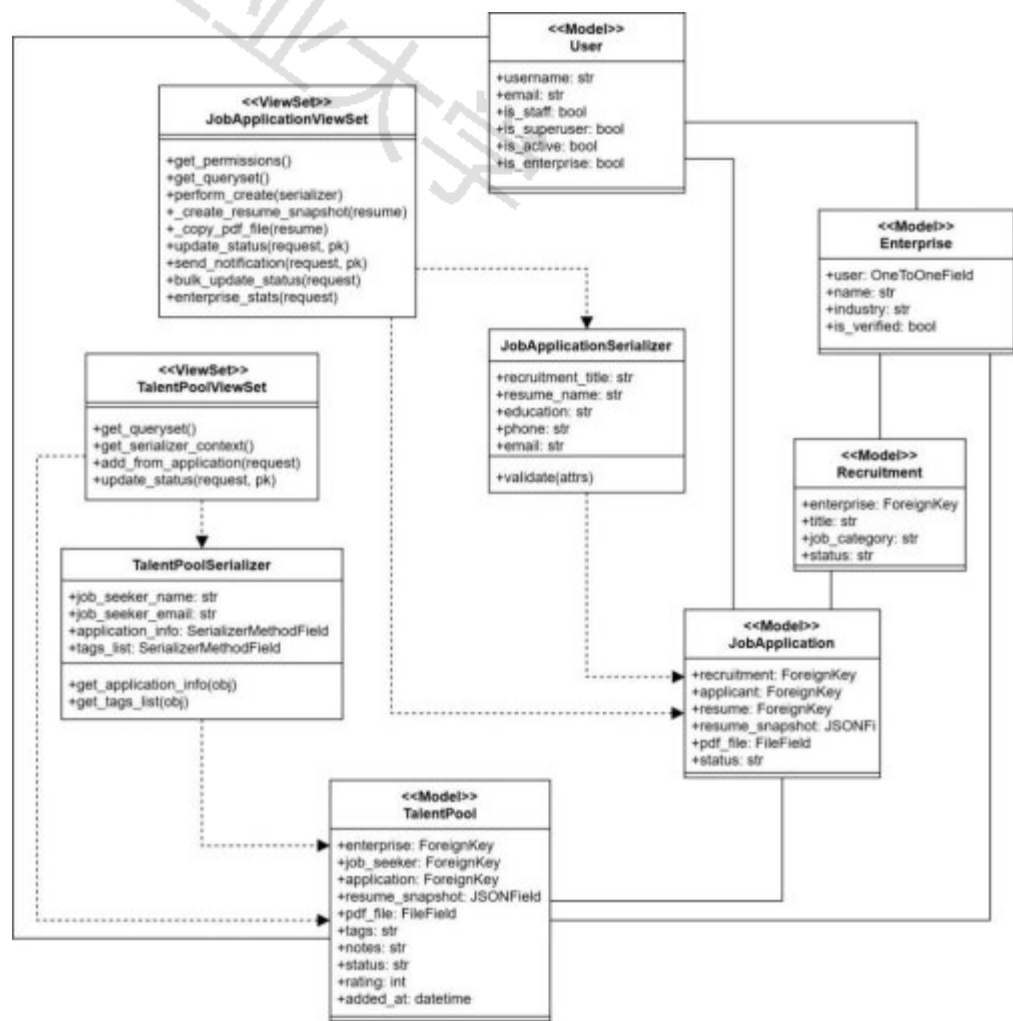


图5.17 企业端招聘过程管理模块类图

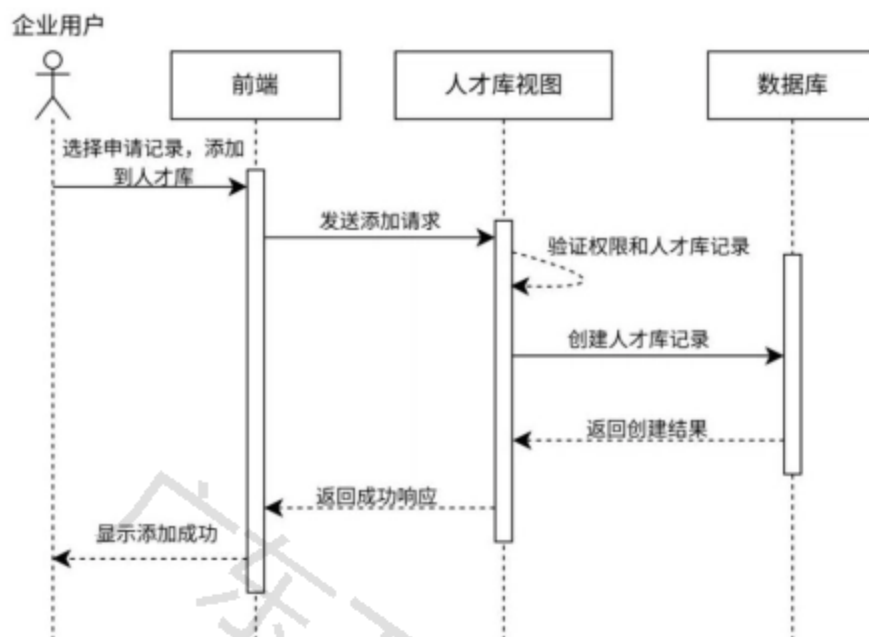


图5.18 人才库建设功能时序图

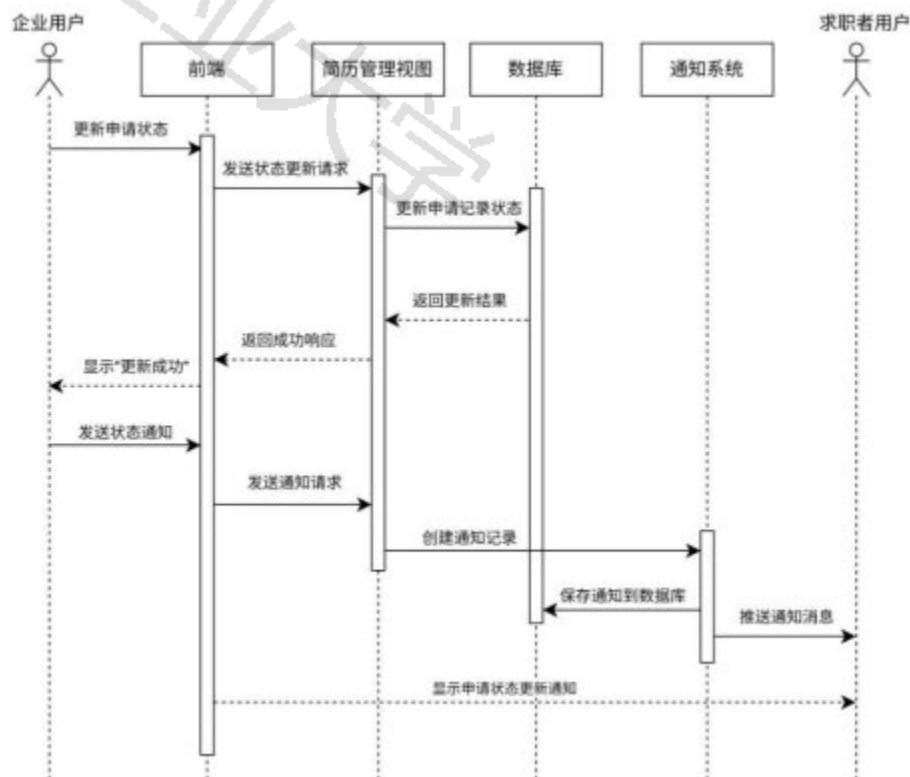


图5.19 笔试面试邀约与进度跟踪功能时序图

5.2.9. 招聘数据统计模块

企业在查看招聘统计数据时，前端发送GET请求，后端调用JobApplicationViewSet的enterprise_stats()方法验证企业身份后返回两类数据：按状态统计各申请状态的数量和一些招聘统计信息。企业查看人才库统计时，前端发送GET请求到TalentPoolViewSet获取人才库记录，前端计算笔试数、面试数、录用数、淘汰数和面试通过率，企业首页还展示综合统计指标，包括正在招聘数、收到简历数、人才库数、已拒绝数和初筛通过率。

统计功能只涉及身份验证与简单计数，实现方法和流程较简单，这里不作时序图的呈现，招聘数据统计模块类图如图5.20所示。

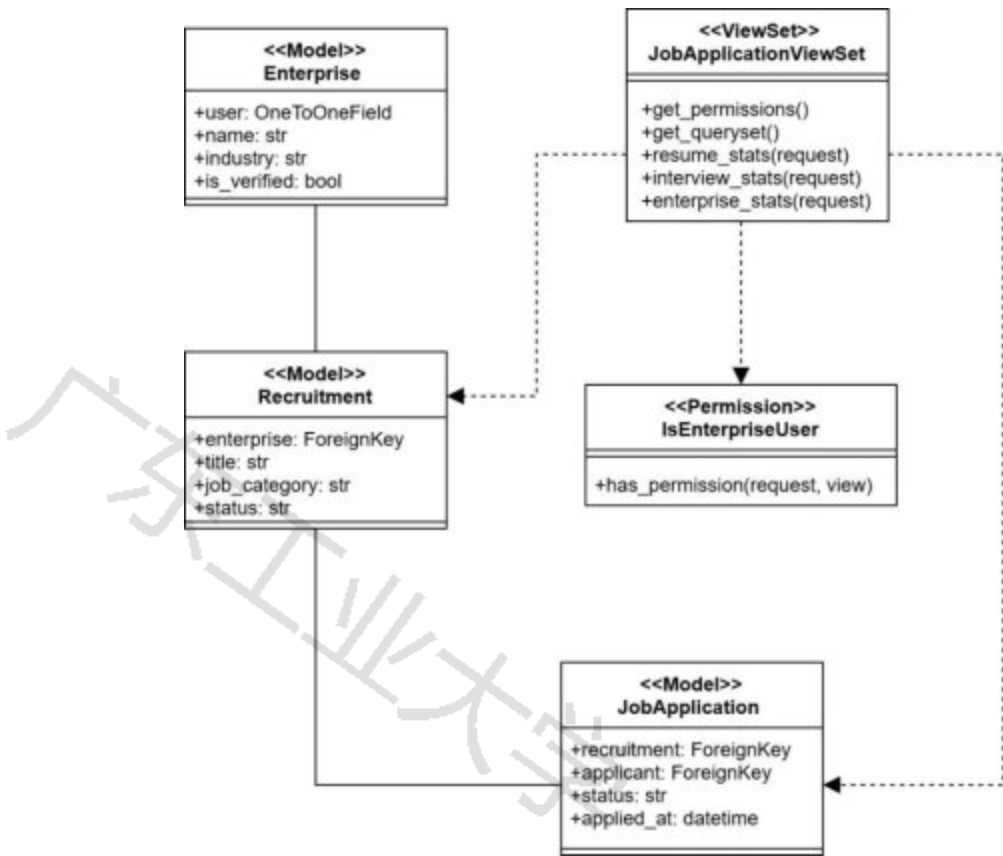


图5.20 招聘数据统计模块类图

企业端的实时聊天室模块、消息通知模块实现思路和设计与普通求职者用户端相似，在用户端模块详细设计部分有做说明，这里不再给出实现思路和说明图。

5.2.10. 管理员模块

1. 用户资质审核：管理员在Django Admin后台的“用户信息”分组下管理实名认证申请，查看所有待审核、已通过、已拒绝的认证记录。审核通过：管理员在认证申请详情页点击“通过”按钮，后端将认证状态设为通过，记录审核人和审核时间，同时发送通知给用户。审核不通过：管理员填写拒绝原因，后端将认证状态设为拒绝，记录拒绝原因，同时发送通知给用户。用户通过认证后，`is_verified`字段为true，可以正常发布文章或招聘信息。
2. 权限分配管理：管理员在系统后台的“用户信息”页面管理用户权限，可以修改用户的`is_staff`字段赋予管理员权限，修改`is_active`字段控制账户激活或冻结，冻结后的用户无法登录系统。
3. 招聘信息审核：企业用户发布的招聘信息默认可以直接发布，不需要管理员审核，只需要通过敏感词的机器检测。企业用户在发布招聘信息时，如果勾选“立即发布”，则招聘信息的状态直接设为PUBLISHED，对所有求职者可见；如果未勾选，则为DRAFT状态，管理员可以在后台查看所有招聘信息，并且可以主动进行信息的下架删除，但当前没有强制审核流程。
4. 文章审核：管理员在后台审核求职者文章，审核通过：标记文章为已通过，文章在前台对所有用户可见。审核不通过：标记文章为已拒绝，同时调用`create_notification`给文章作者发审核结果通知，需要说明的是，文章审核也

是默认通过的，只在文章收到举报后管理员手动审核，管理员也可以主动对已发布的文章进行上下架的处理。

5. 敏感词过滤：管理员在Django Admin后台管理敏感词（添加、删除和查看），前端在发布内容时发送GET请求到check_content接口，后端检测内容是否包含敏感词并返回检测结果，用户发布文章或招聘信息时，前端会调用检测接口，如果内容包含敏感词则阻止发布并提示用户修改。

6. 举报信息处理：管理员在后台处理用户举报，后端返回所有待处理举报记录。管理员可以批量或单独处理举报。举报通过：管理员将举报状态更新为approved，系统会调用create_notification给举报人和被举报人分别发送处理结果通知。举报不通过：管理员将举报状态更新为rejected，填写不通过原因，系统会调用create_notification给举报人发送处理结果通知。

7. 业务数据统计：管理员在Django Admin后台的“用户信息”页面查看业务统计，后端在register/admin.py的UserAdmin.changelist_view方法中统计各类业务数据。统计内容包括：总用户数、活跃用户数、企业用户数、普通用户数、总招聘信息数、已发布招聘信息数、总文章数、已发布文章数、已投递简历数。统计数据在用户列表页面顶部以卡片形式展示。

以上是管理员端主要的功能设计及其基本思路，因为Django Admin是Django框架自带的管理员后台系统，本平台只是在其基础上稍加修改和添加统计模块，这里不呈现详细类图及和时序图说明。

5.3. 本章小结

本章详细先介绍了开发大学生就业信息共享平台所用到的技术，并且结合第四章的内容，详细阐述了平台的用户端、企业端、管理员端主要功能模块的实现思路，并给出了核心的UML类图和功能时序图。

6. 系统测试

本章将对大学生就业信息共享平台进行测试，重点是对该系统的各个功能进行测试以及评估，通过各种测试用例，尽量详尽地对系统功能进行全面功能测试，给出测试结果与效果图，保证平台实现最开始所提到的需求，达到本系统的预期目标。

6.1. 系统的测试环境

大学生就业信息共享平台测试的软硬件环境如表6.1所示。

表6.1 平台测试软硬件环境展示表

软硬件名称	版本信息	备注信息
处理器	12th Gen Intel(R) Core(TM) i5-12500H (3.10 GHz)	开发和测试环境
内存	16GB	运行内存
操作系统	Windows 11	64位操作系统
显卡	NVIDIA GeForce RTX 3050	显卡
Python	3. x	后端开发语言
Django	5. 2. 6	Web框架
Django REST Framework	3. 16. 1	API开发框架

MySQL	8.0.26	数据库
Redis	6.2.6	缓存数据库
Vue.js	3.5.18	前端框架
Node.js	22.12.0	JavaScript运行环境
Naive UI	2.34.4	UI组件库
Microsoft Edge	最新版	测试浏览器

6.2. 系统功能测试

本节根据系统设计的功能模块，对大学生就业信息共享平台的主要功能进行测试，主要采用黑盒测试方法，验证系统各项功能是否达到预期目标。

6.2.1. 用户注册登录功能模块测试

测试用例1：用户注册功能

主要测试企业用户和普通用户的注册功能是否正确实现，管理员未设置单独注册功能，根据Django框架自带的管理员后台，预先设置好一位超级管理员负责全部管理工作，因此不考虑管理员用户注册。表6.2为用户注册测试用例表，图6.1为个人用户注册界面效果图，图6.2为企业注册界面效果图，图6.3为邮箱被注册的提示图。

表6.2 用户注册测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
普通用户注册	1. 输入用户名、邮箱、密码	注册成功，收到验证邮件，数据库中创建用户记录	注册成功，收到验证邮件	通过
	2. 点击注册按钮			
	3. 检查邮箱验证邮件			
企业用户注册	1. 输入企业信息、邮箱、填写验证码	注册成功，但是未实名状态	注册成功，但不能发布招聘	通过
	2. 点击注册			
重复用户名注册	1. 使用已存在的用户名注册	提示用户名已存在，注册失败	提示用户名已存在	通过
	2. 点击注册按钮			
密码强度验证	1. 输入弱密码（如123456）	提示密码强度不足，要求重新输入	提示密码强度不足	通过
	2. 点击注册按钮			





图6.1 个人用户注册界面效果图



图6.2 企业注册界面效果图





图6.3 邮箱被注册提示效果图

需要说明的是：一个邮箱只能绑定一个账号，因此不使用真实邮箱进行功能测试，而是直接向未重复的有效邮箱直接发送验证码，验证码在点击发送后会直接在界面提示弹窗中展示，且验证码有效次数为一次。

测试用例2：用户登录功能

主要测试登录功能的实现和出现错误时的前端提示，表6.3为用户登录功能测试用例表，图6.4为登录失败提示示意图，企业登陆失败提示相同，这里不单独再展示重复效果图。

表6.3 用户登录功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
正确账号密码登录	1. 输入正确的用户名和密码 2. 点击登录按钮	注册成功，收到验证邮件，数据库创建用户记录	注册成功，收到验证邮件	通过
错误密码登录	1. 输入正确用户名和错误密码 2. 点击登录	提示用户名或密码错误	提示用户名或密码错误	通过
JWT令牌过期处理	1. 登录后等待24小时 2. 刷新页面	自动跳转到登录页面，提示令牌过期	自动跳转到登录页面	通过



图6.4 登录失败提示示意图

6.2.2. 简历管理模块测试

测试用例3：简历创建与编辑

主要测试普通求职者用户登录后的简历创建与编辑等管理简历功能，表6.4为简历管理测试用例表，图6.5为简历编辑页面效果呈现图，图6.6为简历列表页面效果呈现图。

表6.4 简历管理测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
创建文字简历及上传PDF简历	1. 填写教育经历、工作经历等基本信 息，并且选择PDF文件上传 2. 保存简历	简历创建成功，数据保存 到数据库	简历创建成功， PDF简历路径被保 存	通过
编辑简历	1. 选择已创建的简历 2. 修改部分信息，以及重新上传新简历信息更新成功，新的 PDF简历 3. 保存简历	PDF路径被保存	简历更新成功	通过
删除简历	1. 选择简历 2. 点击删除按钮	简历删除成功，数据库中 记录被删除	简历删除成功	通过



图6.5 简历编辑页面效果呈现图



图6.6 简历列表页面效果呈现图

6.2.3. 经验分享社区模块测试

测试用例4：文章发布与管理

主要对社区模块的文章和经验贴创作功能进行全面的测试，确保系统关键功能的实现，表6.5为经验分享社区模块测试用例表，图6.7为社区文章创作编辑页面图，图6.8为个人主页管理已发布文章页面图，图6.9为社区文章展示页面效果图，图6.10为文章详情及点赞收藏效果图。

表6.5 为经验分享社区模块测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
发布文章	1. 在导航栏“创作/发布”入口进入编辑页面	文章发布成功，显示在社区列表	文章发布成功，社区内有显示	通过
保存草稿	2. 使用Markdown编辑器编写文章 3. 发布文章			
	1. 编写文章内容 2. 点击保存草稿	文章保存为草稿状态，再次进入文章创作页面可以继续编辑	文章草稿保存成功	通过
编辑已发布文章	1. 在个人主页的“我的发布”下选择已发布文章 2. 点击“更新”按钮 3. 在编辑页面修改内容后点击“更新”	文章内容更新成功	文章内容更新成功	通过
下架文章	1. 在“我的发布”下选择已发布的文章 2. 点击“下架”	文章下架后状态变为草稿	文章成功下架，在草稿箱可见	通过
删除文章	1. 在“我的发布”下选择已发布文章 2. 点击“删除”	文章删除成功，数据库中记录被删除	文章删除成功	通过
文章点赞与收藏	1. 在文章详情页面点击文章底部的点赞按钮和收藏按钮	点赞数和收藏数增加1，按钮状态变为已点赞和已收藏，数据库同步更新	点赞数和收藏数增加1	通过



图6.7 社区文章创作编辑页面图



图6.8 个人主页管理已发布文章页面图





图6.9 社区文章展示页面效果图



图6.10 文章详情及点赞收藏效果图

测试用例5：评论区功能测试

主要测试文章评论区的评论功能是否完成预期，表6.6为评论区功能测试用例表，图6.11为评论区回复功能示意图，图6.12为评论区点赞及更多功能示意图。

表6.6 评论区功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
发表评论	1. 在文章下方评论区输入评论内容 2. 点击“发表评论”	评论显示在评论区	评论显示成功	通过
回复评论	1. 点击某条评论的回复按钮 2. 输入回复内容 3. 提交回复	回复显示在该评论下方	回复显示成功	通过
	1. 选择自己发布的评论	评论删除成功，若是顶级评	评论删除成功，数	

删除自己的评论	2. 点击删除按钮	论，那么二级评论也同步删除	数据库相关评论记录也被删除	
删除自己发布的文章下的评论	1. 以文章作者身份进入文章 2. 选择评论 3. 点击删除按钮	评论删除成功	评论删除成功	通过
评论点赞	1. 点击评论点赞按钮	评论点赞数加一	评论点赞数加一	通过
评论举报	1. 点击评论处的举报按钮 2. 填写举报理由 3. 提交举报	弹出待审核提示，管理员页面收到举报提示	弹出提示，管理员收到举报信息	通过

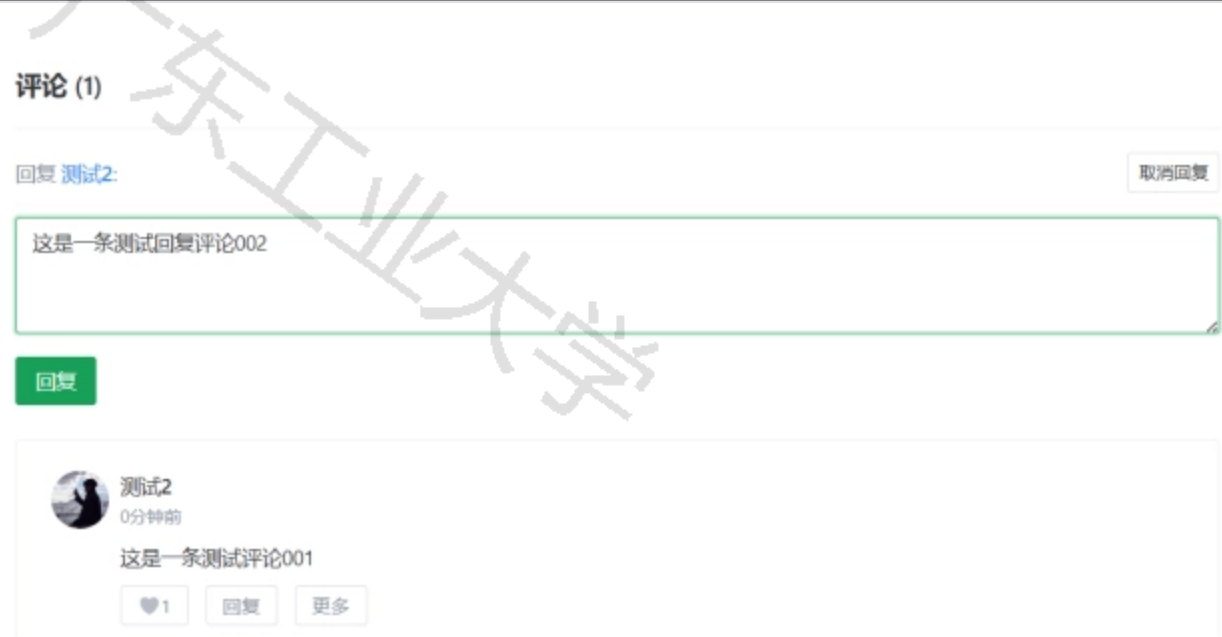


图6.11 评论区回复功能示意图



图6.12 评论区点赞及更多功能示意图

6.2.4. 职位筛选与投递模块测试

测试用例6：职位筛选功能

主要测试职位筛选查看功能是否符合预期，表6.7为职位筛选功能测试用例表，图6.13为招聘信息界面效果图，图6.14为职位详情界面效果图。

表6.7 职位筛选功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
多条件筛选	1. 选择职位类型、地点、薪资范围等 筛选框	显示符合筛选条件的职位	显示符合筛选条件的职位	通过
查看职位详情	1. 点击职位卡片 2. 查看职位详情信息	显示完整职位信息	显示完整职位信息	通过
收藏职位	1. 点击职位收藏按钮	职位添加到收藏列表	职位添加到收藏列表	通过



图6.13 招聘信息界面效果图





图6.14 职位详情界面效果图

测试用例7：职位投递功能

主要测试系统关键功能，即职位投递功能是否正常运行，是否能结合通知模块达到系统预期。表6.8为职位投递功能测试用例表，图6.15为选择简历投递窗口图，6.16为已投递简历查看界面图。

表6.8 职位投递功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
投递简历	1. 选择职位并选择	投递成功，状态为待查看	投递成功	通过
查看投递状态	1. 进入投递记录页面 2. 查看投递状态	显示所有投递记录及状态	显示投递记录	通过
撤回投递申请	1. 选择未处理的投递记录 2. 点击撤回申请	申请投递记录被撤回	企业端申请记录消失，用户端投递记录被删除	通过



图6. 15 选择简历投递窗口图

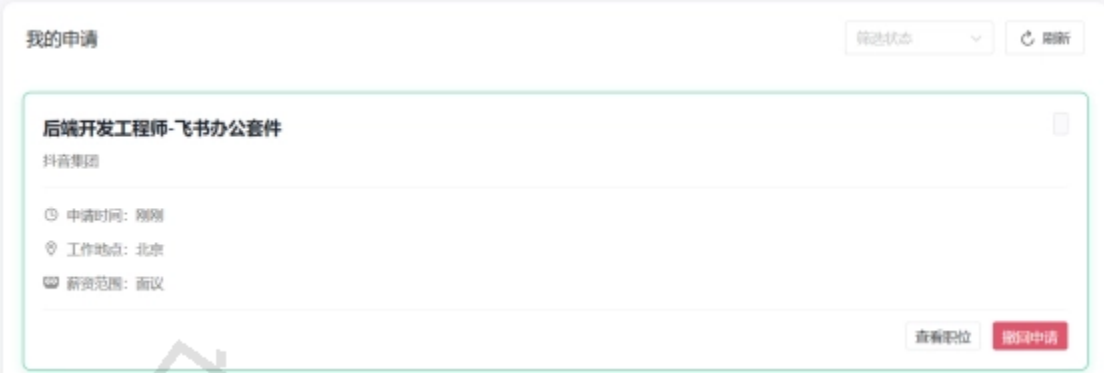


图6. 16 已投递简历查看界面图

6. 2. 5. 聊天室模块测试

测试用例8：实时聊天功能

主要测试沟通聊天板块是否能达到系统预期，表6. 9为实时聊天室测试用例表，图6. 17为实时聊天室窗口图。

表6. 9 实时聊天室测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
发送文本消息	1. 选择聊天对象 2. 输入文本消息 3. 点击发送	消息发送成功，对方实时收到	消息发送成功	通过
发送图片	1. 点击聊天框内的图片图标 2. 选择图片 3. 点击发送	图片发送成功，双方可以放大查看	图片发送成功	通过
发送文件	1. 点击文件上传按钮 2. 选择文件 3. 点击发送	文件上传成功，对方可下载	文件发送成功	通过
消息已读状态	1. 发送消息 2. 对方查看消息	消息状态更新为已读	消息状态更新为已读	通过
消息历史记录	1. 进入聊天室 2. 滚动查看历史消息	显示所有历史聊天记录	显示历史记录	通过

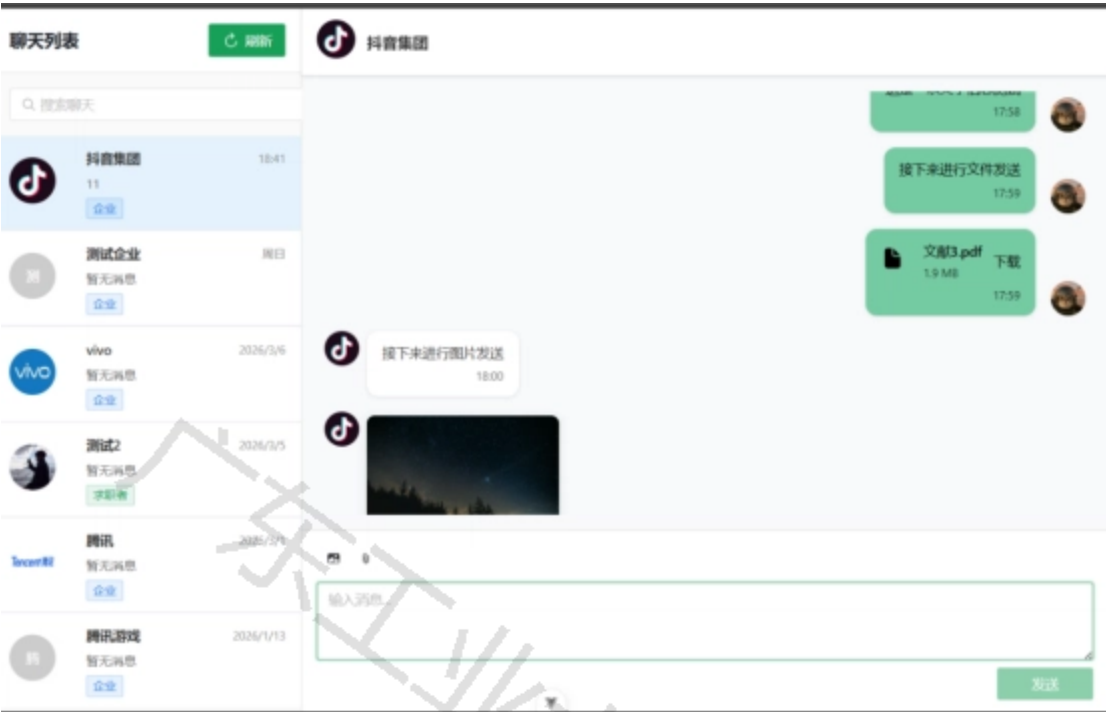


图6. 17 实时聊天室窗口图

6. 2. 6. 通知模块测试

测试用例9：通知功能测试

主要测试普通用户端、企业端、管理员端的实时通知模块是否达到系统预期，表6. 10为通知模块测试用例表，图6. 18为用户通知列表图，图6. 19为用户通知页面列表二图。

表6. 10 通知模块测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
投递通知	1. 学生投递简历 2. 企业查看通知	消息发送成功，对方实时收到	消息发送成功	通过
投递流程更新通知	1. 企业发送招聘流程信息 2. 学生查看通知	学生用户端收到招聘状态更新通知	学生用户端收到招聘状态更新通知	通过
评论回复通知	1. 他人回复评论 2. 查看通知	收到被回复通知详情	收到被回复通知详情	通过
点赞收藏关注通知	1. 他人点赞文章或评论、收藏文章或被关注 2. 查看通知	收到反馈通知	收到反馈通知	通过
举报受理通知	1. 举报文章或用户 2. 管理员审核后查看通知	举报受理后收到反馈通知	举报受理后收到反馈通知	通过
删除通知	1. 选择通知 2. 点击删除	通知删除成功	通知删除成功	通过

- 标记通知已读
1. 进入通知列表

通知状态更新为已读

状态更新为已读

通过
2. 点击通知卡片或点击全部标记已读

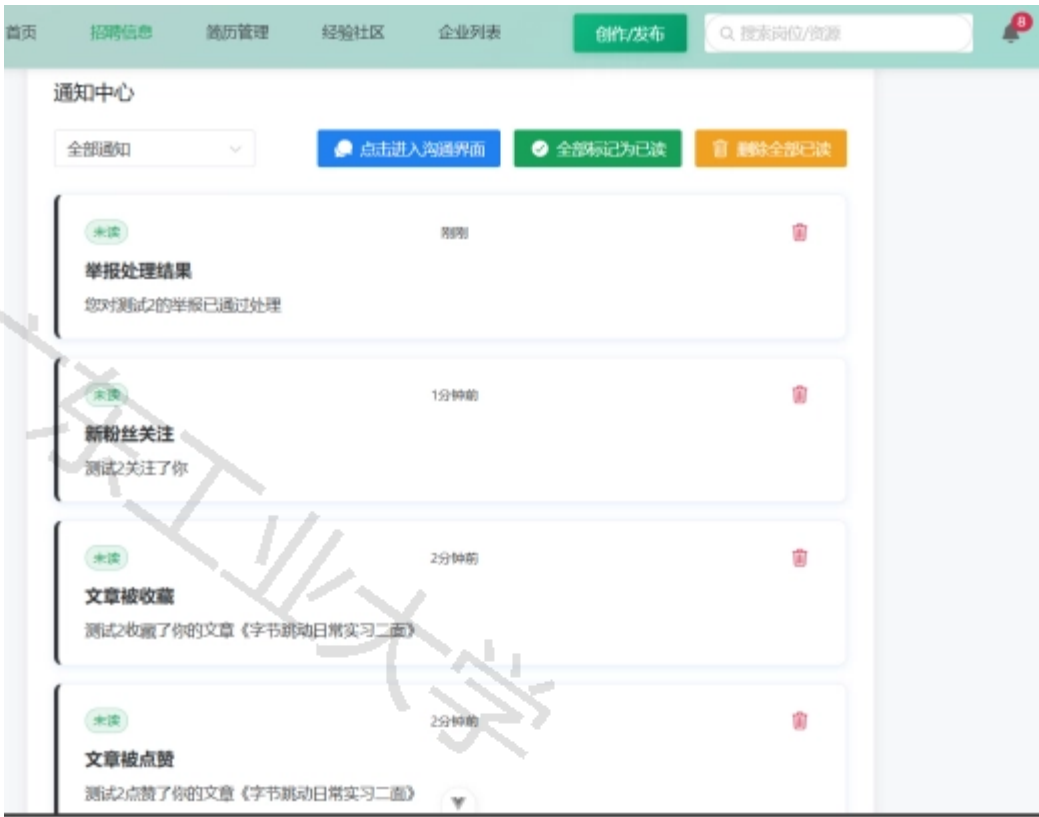


图6.18 用户通知列表图



图6. 19 用户通知页面列表二图

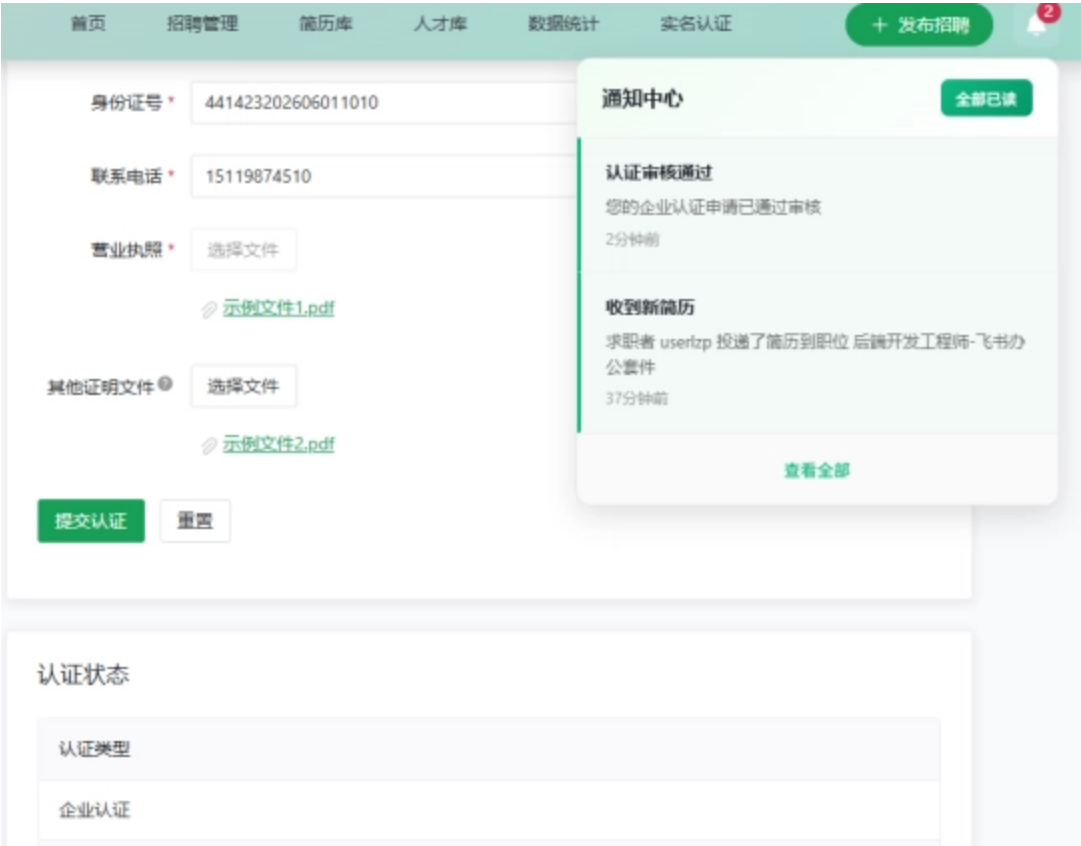
6. 2. 7. 认证功能测试

测试用例10：用户认证功能

主要测试用户实名认证功能是否能够正确实现，实名认证功能关乎用户和企业能否正常使用本系统的服务，表6. 11为用户认证功能测试用例表，图6. 20为企业提交认证信息及审核通知页面效果图。

表6. 11 用户认证功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
普通用户提交认证材料	1. 填写个人实名信息 2. 上传学信网截图等证明材料 3. 提交审核	材料提交成功，状态更新为待审核	材料提交成功	通过
企业用户提交认证材料	1. 填写企业相关信息 2. 上传营业执照等相关证明材料 3. 提交审核	材料提交成功，状态更新为待审核	材料提交成功	通过
查看认证状态	1. 进入实名认证页面 2. 查看认证状态	显示当前审核认证状态	显示认证状态	通过
更新认证信息	1. 修改认证信息和证明材料 2. 保存修改	更新状态为待审核	状态更新为待审核，管理员收到审核提示	通过



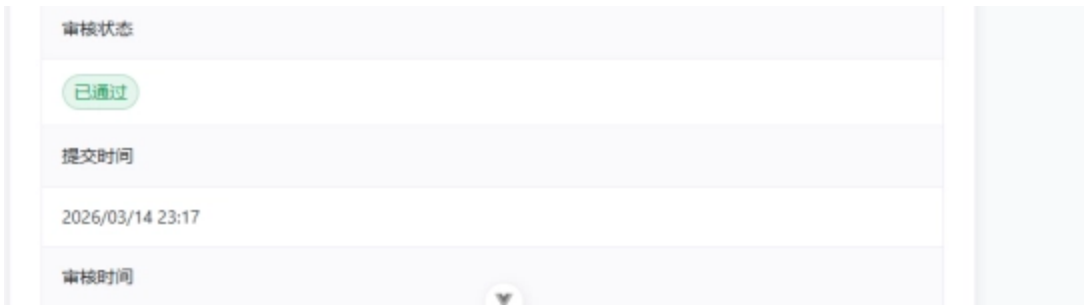


图6.20 企业提交认证信息及审核通知页面效果图

6.2.8. 企业端功能测试

测试用例11：职位发布功能

主要测试企业端职位发布功能是否合理和达到系统预期，表6.12为职位发布功能测试用例表，图6.21为企业端招聘信息发布页面图，图6.22为企业端招聘管理页面图。

表6.12 职位发布功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
发布职位	1. 填写职位信息 2. 勾选立即发布职位并发布	职位发布成功，职位显示在职位列表	职位发布成功	通过
编辑职位	1. 选择并点击编辑已发布职位 2. 修改职位信息 3. 点击保存	职位信息更新成功	职位信息更新成功	通过
下架职位	3. 进入实名认证页面 4. 查看认证状态	显示当前审核认证状态	显示认证状态	通过

应届毕业生

学历要求 *

本科

职位描述 *

团队介绍：字节跳动云基础设施部门，通过云技术管理着百万量级的服务器构成的超大数据中心。我们通过深度优化千万级容器实例与算力优化，搭建EB级数据存储治理体系，探索新一代搜索型数据库与大规模AI集群下的高速网络通信，我们积极拥抱开源和创新的软硬件架构，致力于构建业界领先、稳定、高可用的面向LLM的AI云原生的基础设施架构与产品矩阵，为整个公司的业务和客户发展保驾护航。

1、负责火山引擎AI云原生以及Agent平台产品的产品规划、设计与迭代，调研与分析客户痛点，研究市场需求

任职要求 *

1、2026届获得本科及以上学历，计算机科学与技术、软件工程等相关专业优先；

2、了解LLM的技术原理和进展，熟悉海内外大模型产品，能够阅读和理解相关文档及论文；

3、优秀的文字功底和数据意识，对不同专业和兴趣领域的数据质量有判断力，有深入的产品思考；

4、熟练运用Prompt Engineering、各类大模型的API调用、Agent等模型能力，可以将大模型融入工作流；

5、具有愿景和目标驱动，具备较强交付意识和沟通协作能力，适应快节奏的工作方式；

6、对IT行业的新技术以及云基础设施、AI、信息安全等领域的业界趋势有很强的好奇心，能够持续学习。

截止日期 *

2026-06-01



图 6 . 2 1 企业端招聘信息发布页面图

标题	招聘类型	职位类型	岗位名称	工作地点	薪资	状态	发布时间	操作
AI产品经理-算力与AI基础...	社招	软件开发	AI产品经理-算力与AI基础设...	深圳	面议	已发布	2026/3/14	编辑 下架 删除
测试工程师	社招	测试	测试工程师	广州	20000-30000	已发布	2026/3/14	编辑 下架 删除
后端开发工程师-飞书办公...	校招	软件开发	后端开发工程师	北京	面议	已发布	2026/3/14	编辑 下架 删除

图6.22 企业端招聘管理页面图

测试用例12：招聘过程管理功能

主要测试企业端招聘过程中的管理功能是否实现主要功能，表6.13为企业招聘过程功能测试用例表，图6.23为企业筛选简历界面图，图6.24为企业人才库界面图。

表6.13 企业招聘过程功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
筛选简历	1. 设置筛选条件 2. 点击筛选	显示符合条件的简历列表	显示符合条件的简历	通过
添加到人才库	1. 选择简历 2. 添加到人才库	简历添加到人才库	简历添加到人才库	通过
发送笔面试通知或邀请	1. 人才库中选择合适候选人 2. 选择流程信息并发送邀请	笔面试通知邀请发送成功，学生收到通知	通知邀请发送成功	通过



图6.23 企业筛选简历界面图





图6.24 企业人才库界面图

6.2.9. 管理员功能测试

测试用例13：用户管理功能

主要为管理员管理平台用户功能权限的简单测试，表6.14为管理员端用户管理功能测试用例表，图6.25为管理员后台冻结解冻用户账号示意图。

表6.14 管理员端用户管理功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
查看用户列表	1. 进入用户管理页面 2. 查看用户列表	显示所有注册用户	显示所有用户	通过
冻结用户	1. 选择用户 2. 点击冻结按钮	用户状态更新为已冻结，无法登录	用户状态更新为已冻结，用户无法登录	通过
解冻用户	1. 选择已冻结用户 2. 点击解冻按钮	用户状态更新为正常，用户功能不受影响	用户状态正常	通过

选择 用户 来修改

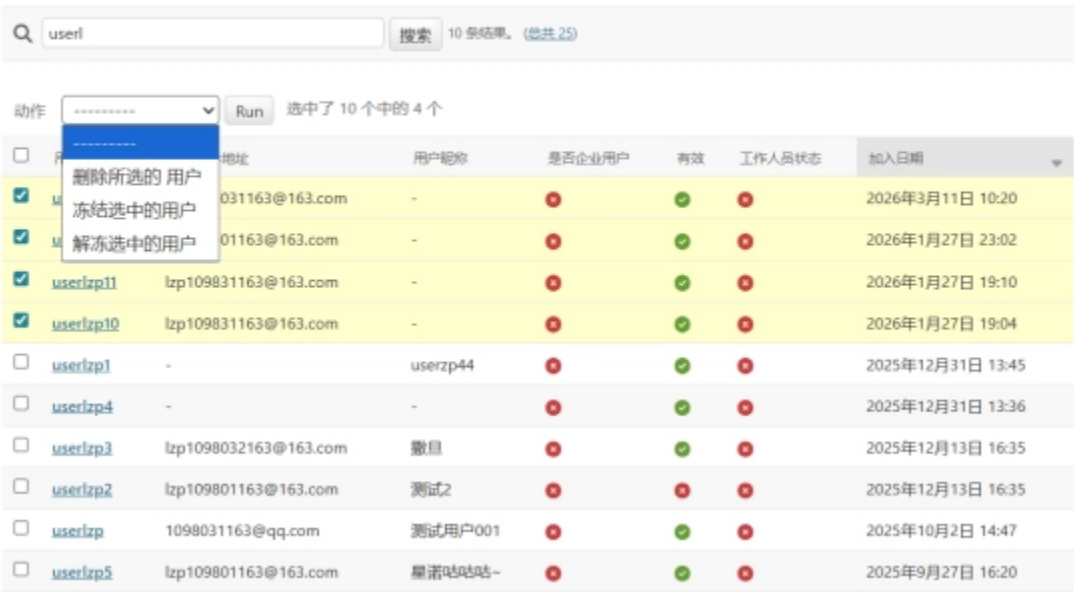


图6. 25 管理员后台冻结解冻用户账号示意图

测试用例14：内容审核功能

主要测试管理员对平台的各种审核功能是否正常实现，表6. 15为管理员内容审核功能测试用例表，图6. 26为管理员后台认证审核效果图，图6. 27为招聘信息界面效果图。

表6. 15 管理员内容审核功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
审核个人用户和企业认证	1. 查看待审核用户和企业用户 2. 查看认证材料	用户端认证状态更新，收到通知	认证状态更新	通过
审核文章	1. 查看被举报的待审核文章 2. 检查文章内容 3. 通过或拒绝审核	文章状态更新，作者收到通知	文章状态更新	通过
处理举报	1. 查看举报记录 2. 查看举报内容 3. 处理举报	举报状态更新，相关用户收到通知	举报处理成功	通过



图6. 26 管理员后台认证审核效果图

选择 举报记录 来修改

Q

搜索

2026

03月7日

03月8日

03月9日

03月14日

03月15日

动作

Run

选中了 7 个中的 0 个

☐	ID	举报人	被举报用户	举报类型	举报原因	处理状态	举报时间
☐	21	user1zp2	user1zp	举报用户	冒充他人	待处理	2026年3月15日 12:20
☐	20	user1zp	user1zp2	举报用户	原因1	已通过	2026年3月14日 22:45

☐	12	userlzp	userlzp2	举报用户	fvsdxc	已通过	2026年3月9日 12:11
☐	16	userlzp	userlzp2	举报评论	谩骂攻击	已通过	2026年3月8日 17:10
☐	14	userlzp	userlzp2	举报用户	对方的分地方	已通过	2026年3月7日 14:38
☐	3	testuser2	userlzp5	举报用户	测试拒绝举报	已通过	2026年3月7日 13:48
☐	1	testuser2	userlzp	举报用户	测试举报自己	待处理	2026年3月7日 13:39

7 举报记录

图6. 27 招聘信息界面效果图

测试用例15：数据统计功能

主要测试管理员端对整个平台数据的统计功能能否正确显示，表6. 16为管理员端数据统计功能测试用例表，图6. 28为管理员后台统计界面图。

表6. 16 管理员端数据统计功能测试用例表

测试项目	测试步骤	预期结果	测试结果	测试结论
查看用户统计	1. 进入数据统计页面 2. 查看用户数据	显示用户注册数、活跃用户数等	显示用户统计数据	通过
查看职位统计	1. 查看职位数据 2. 查看投递数据	显示职位发布数、投递数等	显示职位统计数据	通过
查看社区统计	1. 查看社区数据	显示文章数、评论数等	显示社区统计数据	通过

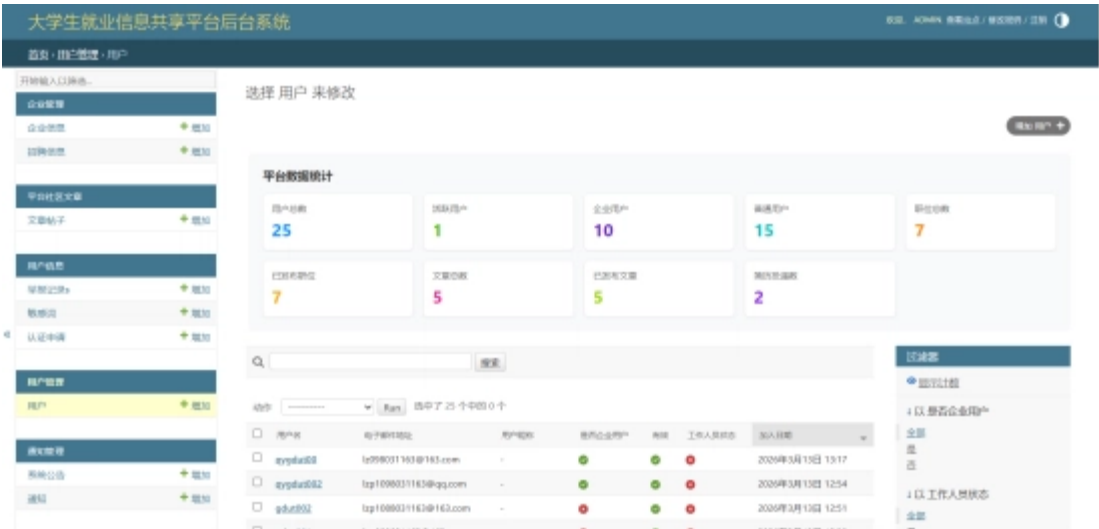


图6. 28 管理员后台统计界面图

6.2.10. 三端主界面展示

图6. 29为普通用户端首页，图6. 30为企业端首页，图6. 31为管理员后端用户首页。



图6.29 普通用户端首页

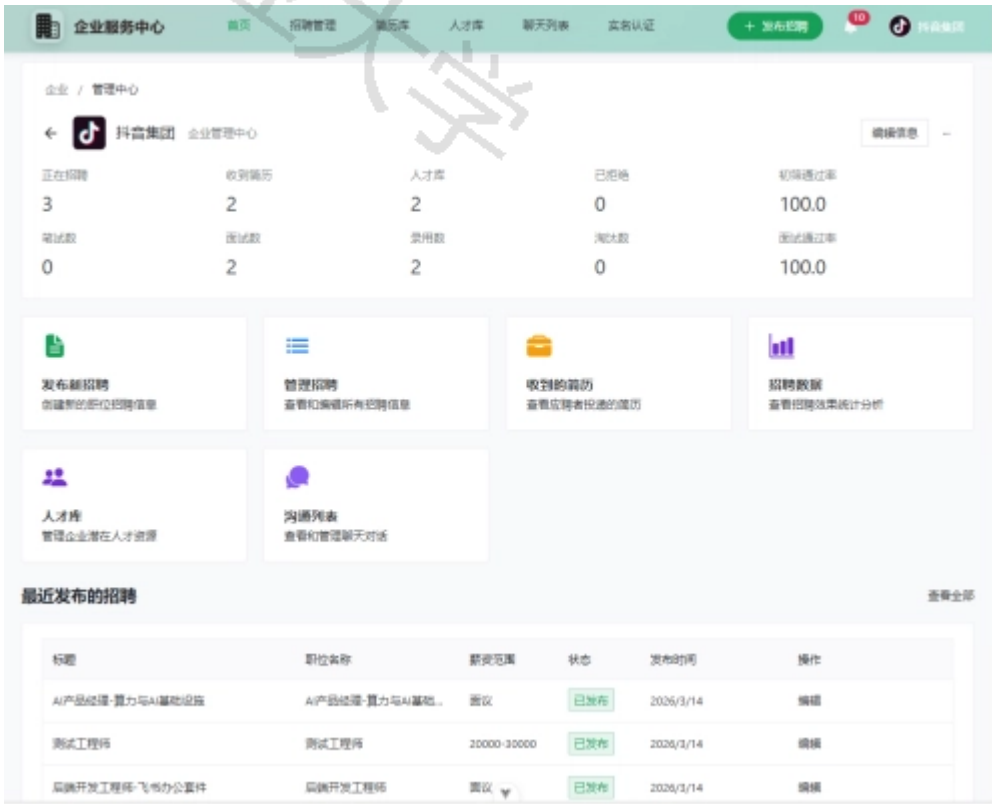


图6.30 企业端主界面首页

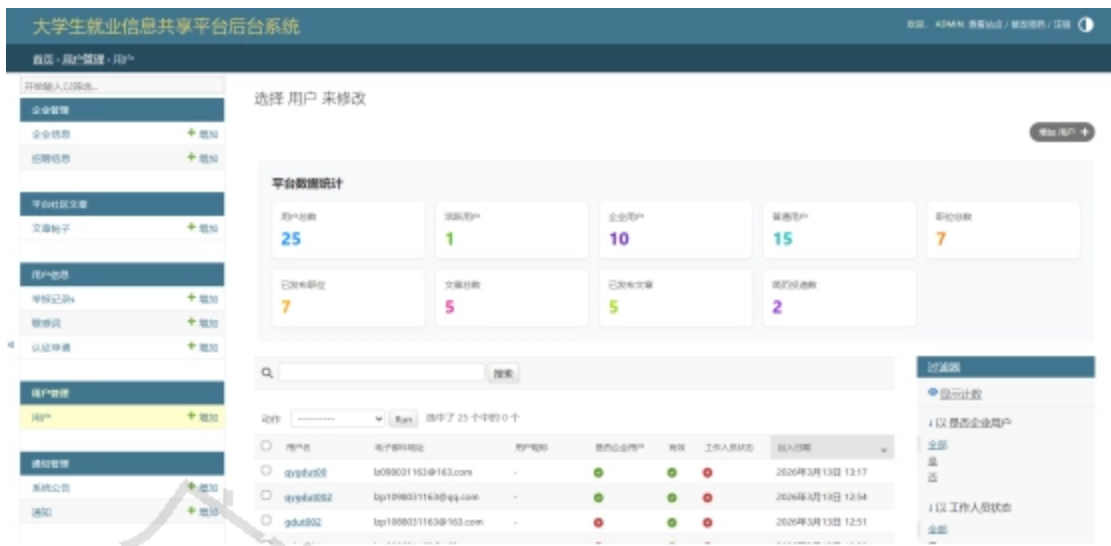


图6. 31管理员后端用户首页

6. 3. 本章小结

本章先介绍了大学生就业信息共享平台测试的软硬件环境，采用黑盒测试的方法，针对学生端、企业端和管理员端的核心功能模块设计测试用例，完成各端全流程的功能测试和交互测试，各模块均能按照预期正常实现，也为各种异常场景设置了提示和处理机制，为后续的实际落地提供了可靠的依据，美中不足的是本平台尚未详细对性能以及安全性进行深入开发和测试，后续将对本平台的安全性、可靠性和性能效率进行更深入的优化。

总结与展望

本文基于Django框架和Vue.js前端技术，设计并实现了一个功能相对完整的大学生就业信息共享平台，系统使用的是前后端分离架构，后端使用Django REST Framework提供RESTful API接口，前端使用Vue.js构建交互界面，结合MySQL数据库和Redis缓存，实现了学生、企业和管理员三类用户的需求。

在系统开发过程中，先完成需求分析和总体设计，明确需求和模块划分，并使用Django REST Framework构建后端API接口，设计数据库存储用户、简历、招聘职位等核心数据，并且使用Vue.js开发前端页面，最后进行功能模块测试和集成测试，测试结果表明，平台能够实现就业信息共享的需求与企业招聘管理的核心需求。

本大学生就业信息共享平台集成了WebSocket实时通信技术，实现了企业用户与求职者用户之间的实时聊天和消息推送功能，这是本系统的亮点之一。系统还建立了内容审核机制，包括企业认证审核、社区和招聘信息内容敏感词过滤、内容审核、举报处理等功能，以确保平台内容的真实性和可靠性。企业端实现简历的条件筛选和人才库建设功能以及招聘数据统计功能，以便企业直观了解当前招聘进度和效果。

尽管本系统实现了预期的主要功能，但也有不足之处。本系统未经过较大数据量和高并发情况下的性能测试，数据库查询性能有很大提升空间，而且安全漏洞的防护措施并不够完善。对于移动端的适配也需要进一步优化，前端交互体验也未能进行深度地优化。另外，系统虽然尽可能实现了核心功能，但在一些细节方面不够完善，例如简历模板功能、职位推荐算法、智能匹配等功能尚未实现。

因此，本系统可以从多个方面进行优化和扩展。比如可引入机器学习算法以实现职位推荐和简历匹配功能，集成视频通话功能，支持企业进行远程视频面试，对接学信网和企业资质认证相关网站进行用户资质的详细认证，完善前端交互体验以及开发或适配移动端版本，使用户能够随时随地使用平台功能。另外，还可以进一步完善系统的

安全防护机制, 包括加强输入验证、实现更严格的密码策略、增加双因素认证等。在性能提升方面, 可以引入负载均衡、数据库读写分离、CDN加速等技术, 提升系统的并发处理能力和响应速度。

参考文献

- [1]赵永红, 刘慧, 郭传州, 等. 高校大学生智能就业平台的设计与应用[J]. 宜春学院学报, 2025, 47(06):40-43.
- [2]沈祥, 王永贵, 张人镜. 2025年高校毕业生就业新趋势观察及应对策略初探[J]. 河南教育(高教), 2025, (04):53-54.
- [3]张玥. 新媒体视阈下大学生就业指导工作的信息化建设研究[J]. 办公室业务, 2024, (01):39-40+64.
- [4]张子怡. 新媒体应用于大学生就业信息推送的策略研究[J]. 新闻研究导刊, 2023, 14(24):172-175.
- [5]孙文倩, 柴天笑, 王晴, 等. 大学生就业平台模式探究[J]. 合作经济与科技, 2022, (17):140-143. DOI:10.13665/j.cnki.hzjyjkj.2022.17.020.
- [6]Hamam, Muhammad Haznor Haqiff, Siti Zulaiha Ahmad, and Jasmine Seow Tiap Lim. "Web-based internship recruitment system with integration of instant notification service for university students." Journal of Computing Research and Innovation (JCRINN) 10.1 (2025): 25-39.
- [7]Malayil, Shanid, et al. "Smart AI Based Online Platform for Competitive Coding, Technical Interview, Assessments and Recruitment." International Conference on Computer Science and Communication Engineering (ICCSCE 2025). Atlantis Press, 2025.
- [8]Hui, Bowen, Eileen Wood, and Carlos Khalil. "An analysis and evaluation of the design space for online job hunting and recruitment software." International Conference on Human-Computer Interaction. Cham: Springer International Publishing, 2021.
- [9]邱红丽, 张舒雅. 基于Django框架的web项目开发研究[J]. 科学技术创新, 2021(27):97-98. DOI:10.3969/j.issn.1673-1328.2021.27.040.
- [10]贺宗平, 贺曦冉, 秦新国. 一种Python ORM框架性能测试分析方法研究[J]. 现代信息科技, 2021, 5(06):83-86. DOI:10.19850/j.cnki.2096-4706.2021.06.021.
- [11]杨朝升. CSRF攻击技术浅析[J]. 石家庄职业技术学院学报, 2023, 35(04):29-32.
- [12]张晨. Web应用XSS网络安全漏洞分析[J]. 襄阳职业技术学院学报, 2025, 24(04):100-106. DOI:10.20219/j.cnki.2095-6584.2025.04.022.
- [13]王安琪, 杨蓓, 张建辉, 等. SQL注入攻击检测与防御技术研究综述[J]. 信息安全研究, 2023, 9(05):412-422.
- [14]陈承欢. Vue.js基础与应用开发实战[M]. 人民邮电出版社:202211:318.
- [15]曾超宇, 李金香. Redis在高速缓存系统中的应用[J]. 微型机与应用, 2013, 32(12):11-13. DOI:10.19358/j.issn.1674-7720.2013.12.004.
- [16]范展源, 罗福强. JWT认证技术及其在WEB中的应用[J]. 数字技术与应用, 2016, (02):114. DOI:10.19695/j.cnki.cn12-1369.2016.02.087.

致谢

行文至此, 也意味着我的本科生涯即将画上句号。回首这段求学之路, 心中充满感激之情。在此, 我谨向所有关心、支持和帮助过我的老师、同学、家人表示最诚挚的谢意。

我由衷的感谢我毕业设计的指导老师，康培培老师，从毕业设计的选题、开题报告、初稿、内容的多次修改和最后的完成，您在每个阶段都给予了我建议和指导，让我明确方向不断改进，同时也感谢四年学生生涯中遇到的所有老师，感谢你们在学习和生活中给予的所有帮助。

感谢学校为我提供了良好的学习环境和丰富的学习资源，正是在这样优美的校园环境中，在老师的悉心教导和同学们的陪伴下，我才能够茁壮成长，顺利完成学业。

感谢朝夕相伴的同窗与挚友，求学之路，幸得同行，我们在课堂上切磋琢磨，在图书馆并肩奋斗，在困境中相互鼓励，在喜悦时共同分享。虽即将各赴前程，但这份纯粹真挚的情谊我将永远珍藏。

最厚重的谢意，献给我挚爱的家人，你们是最坚实的依靠，也是我最温暖的港湾。一路走来，你们无私的关爱、默默的支持与无条件的包容，为我扫除后顾之忧，让我能够心无旁骛地潜心向学、追逐理想。一粥一饭的温暖，一言一行的牵挂，都是我前行路上最坚定的力量。父母之恩，情深似海，愿家人安康顺遂，岁月无忧。最后，感谢坚守初心的自己。在漫长的求学与论文写作的过程中，有过迷茫与困顿、疲惫与彷徨，但未曾放弃。伏案深耕，反复推敲，在坚持中突破，在磨砺中成长，终不负时光，不负热爱。

学业的终点，亦是人生的起点。前路漫漫亦灿灿，我将怀揣所有感恩与期许，铭记师恩、珍惜情谊、不负家人，以更坚定的信念、更踏实的脚步奔赴前路，以实际行动回馈所有关爱与帮助，不负韶华，不负相遇。

须知：

- 报告编号系送检论文检测报告在本系统中的唯一编号。
- 本报告结合AI大模型自动生成，结果仅供参考。
- 报告支持中、英文内容检测。



微信公众号

唯一官网：<https://vpcs.fanyu.com> | 客服邮箱：vpcs@fanyu.com | 客服热线：400-607-5550 | 客服QQ：4006075550